Attention & Transformers

Michalis Vazirgiannis

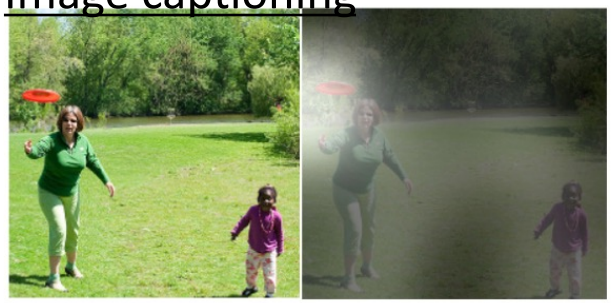
Distinguished Prof. @Ecole Polytechnique, IPP, France

http://www.lix.polytechnique.fr/dascim

October 2026

Attention is ubiquitous in DL today

Image captioning



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Show, attend and tell: Neural image caption generation with visual attention (Xu et al. 2015)

Attention is All You Need - Transformer

TITLE	CITED BY	YEAR
Attention is all you need A Vaswani, N Shazeer, N Parmar, J Uszkoreit, L Jones, AN Gomez, ... Advances in neural information processing systems 30	209415	2017

10/2025

Attention is **all you need** [PDF] neurips.cc

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

… to attend to **all** positions in the decoder up to and including that position. **We need** to prevent … **We** implement this inside of scaled dot-product **attention** by masking out (setting to −∞) …

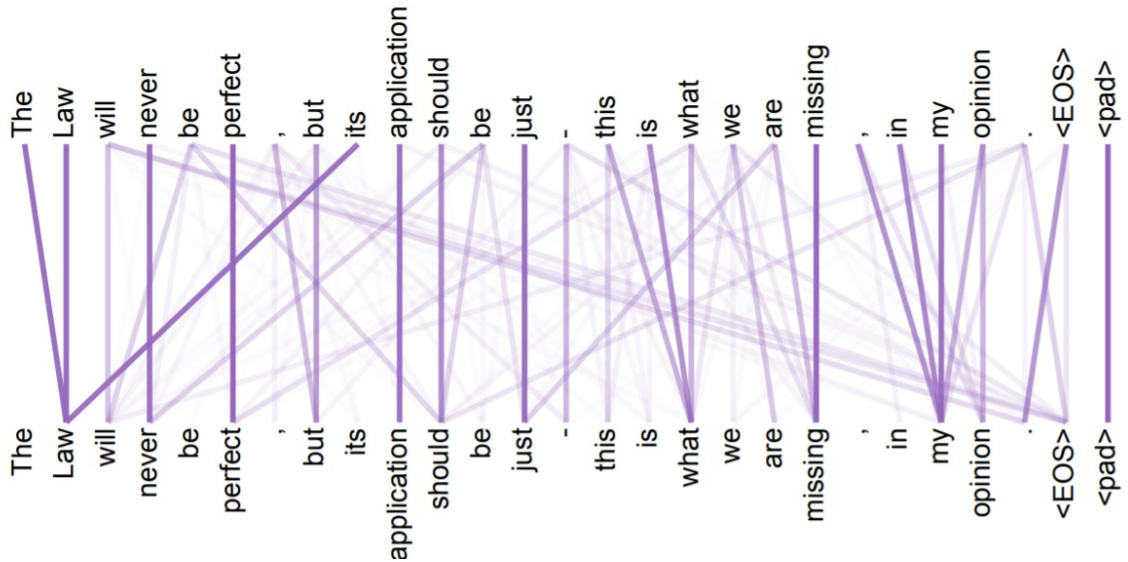
☆ Save 99 Cite Cited by 213142 Related articles All 70 versions »

01/2026

Attention 👀

"The computer which I had just put into the machine room on the fifth floor crashed."

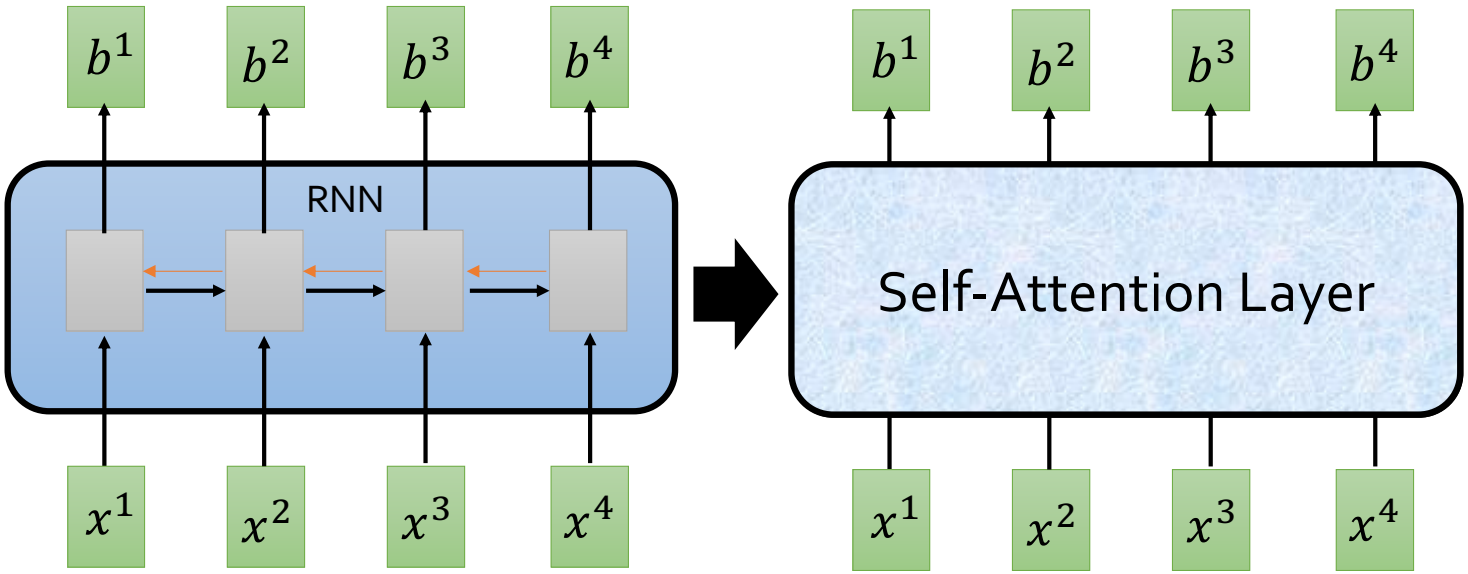
- Core idea: build a mechanism to focus ("attend") on a particular part of the context.
- How can this overcome the "long-range dependencies" problem in RNNs? By allow the model to **directly "look"** 👀 at all tokens and decide which one is useful.



[Attention Is All You Need, Vaswani et al. 2017]

Self-Attention

- b^i is obtained based on the whole input sequence.
- can be parallelly computed.

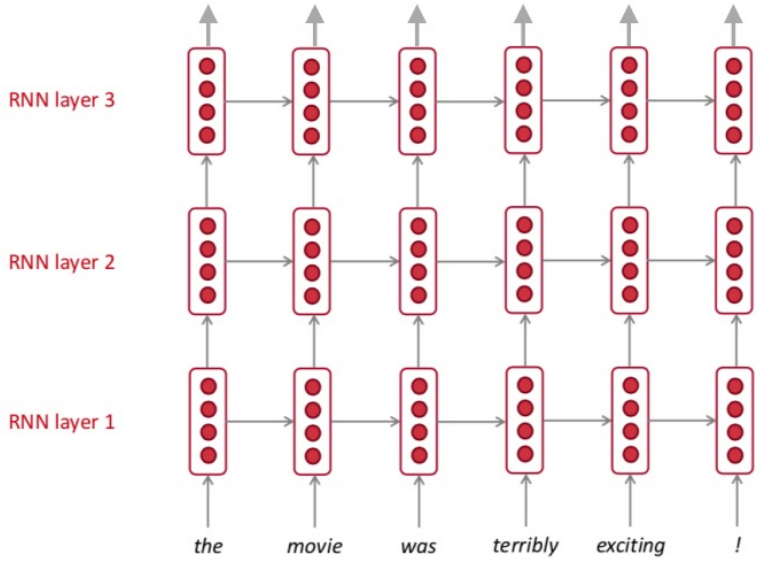


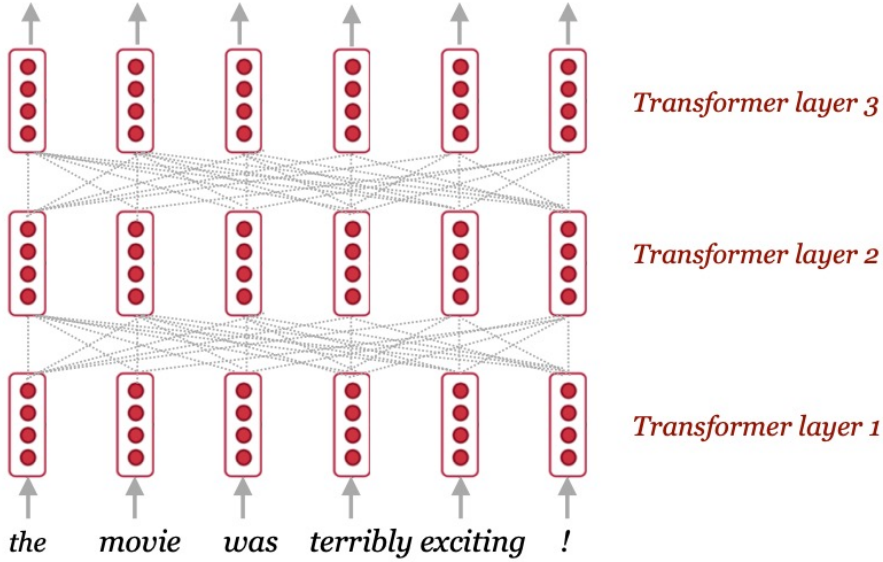
Idea: replace any thing done by RNN with self-attention.

"Neural machine translation by jointly learning to align and translate" Bahdanau etl. 2014;
"Attention is All You Need" Vaswani et al. 2017

RNN vs Transformer

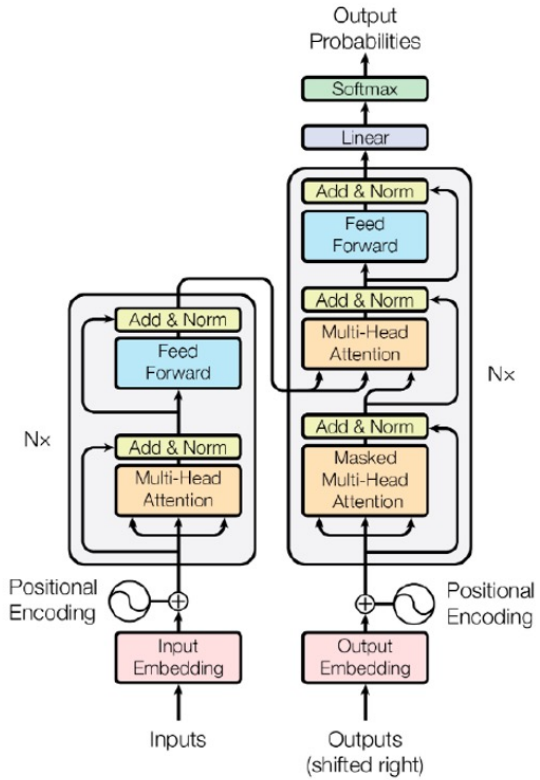
- Notice that self-attention can directly "look" at all tokens and decide which one is useful.





Transformer

- A model that follows the encoder-decoder structure
- input tokens mapped to a sequence of continuous representations
- representations are consumed by the decoder which generates a sequence of outputs.
- Does not use RNNs or CNNs - only uses dense and attention layers.



The weather is nice <cls> Le temp est beau

The weather is nice <cls> Le temp est beau

Defining Self-Attention

- Terminology:
  - Query: to match others
  - Key: to be matched
  - Value: information to be extracted

[Vaswani et al. 2017: https://arxiv.org/abs/1706.03762]

q: query (to match others)

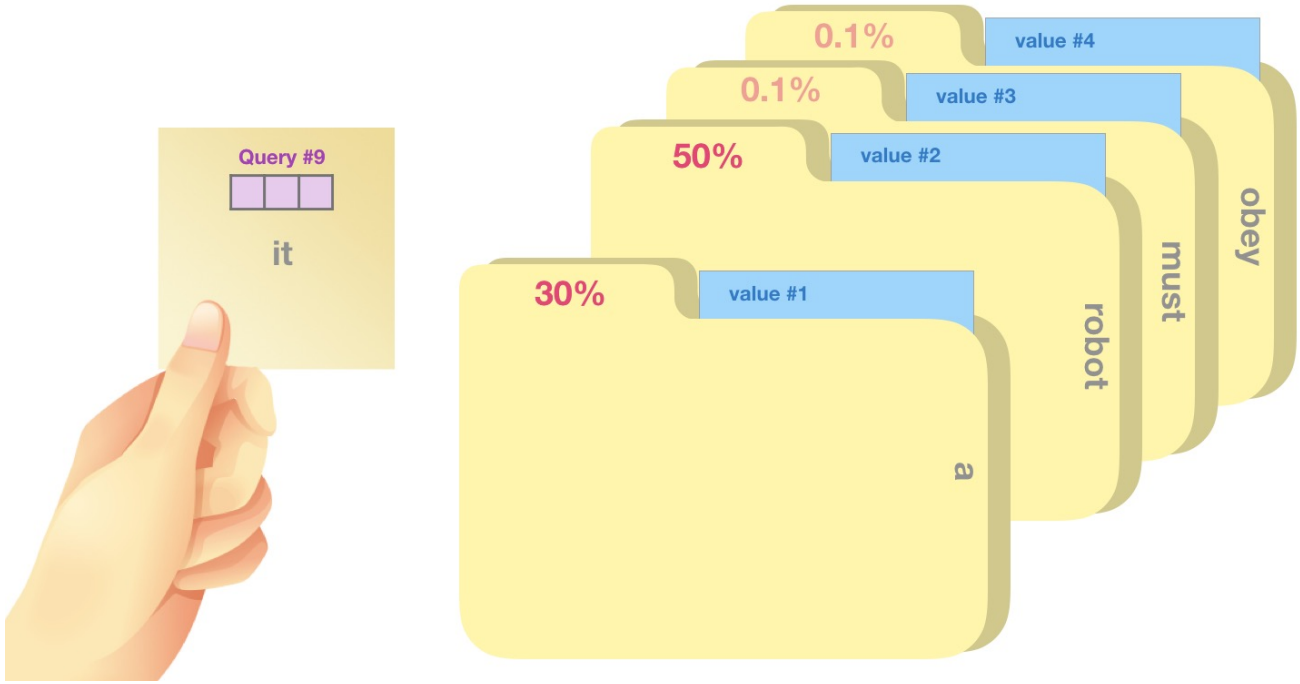
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



q: query (to match others)

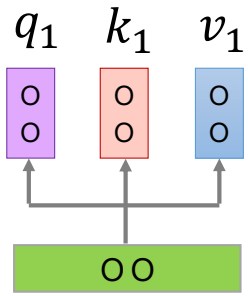
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



The

q: query (to match others)

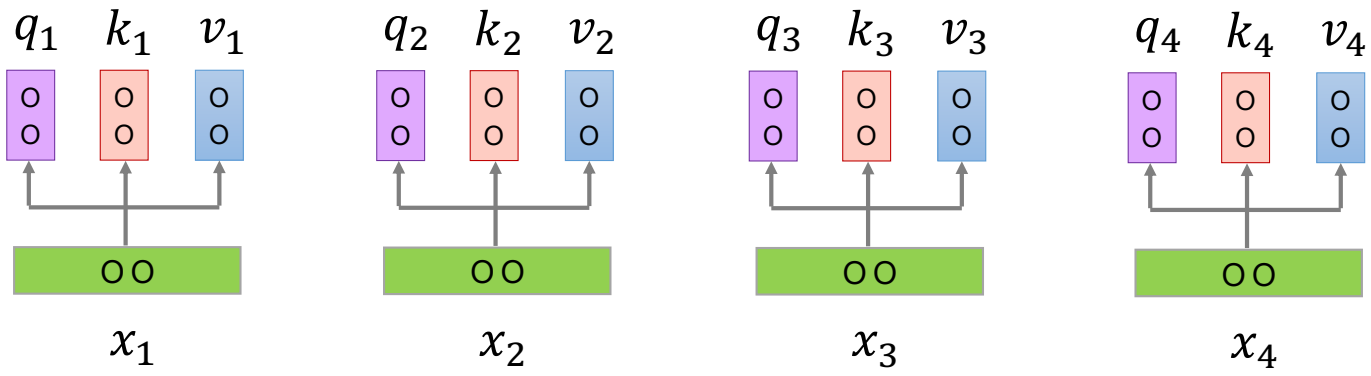
$$q_i = x_i W^q$$

k: key (to be matched)

$$k_i = x_i W^k$$

v: value (information to be extracted)

$$v_i = x_i W^v$$



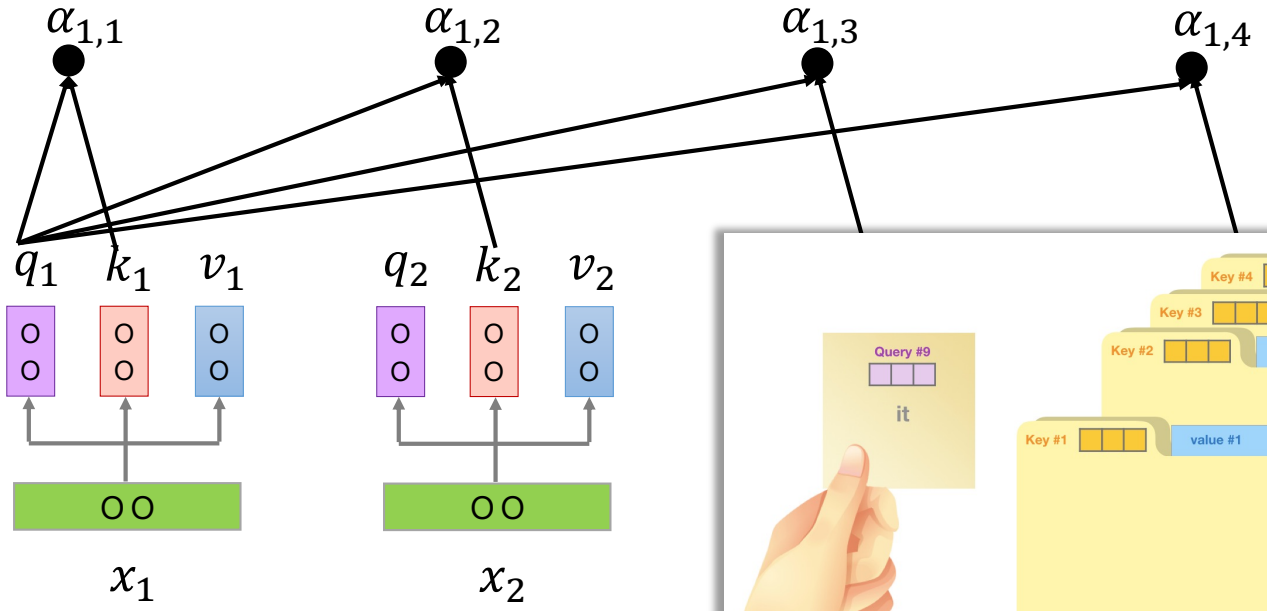
$$\alpha_{1,i} = \underbrace{q^1 \cdot k^i / \sqrt{d}}_{\text{Scaled dot product}}$$

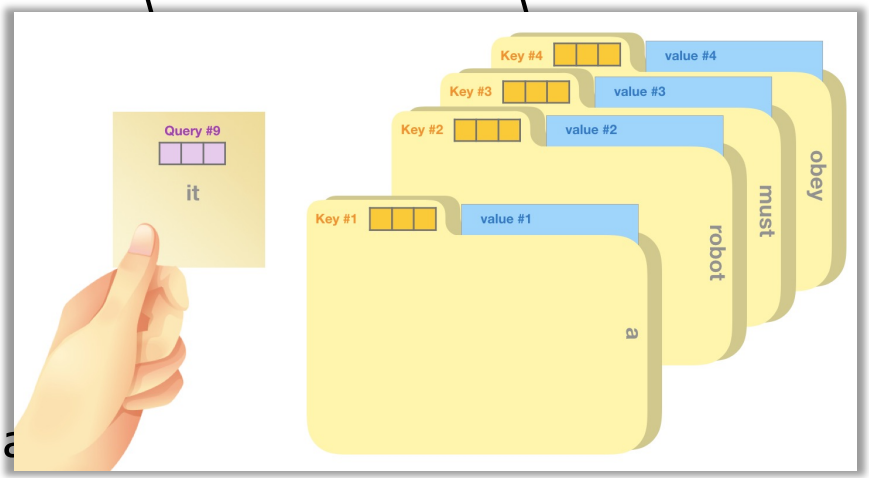
q: query (to match others)

k: key (to be matched)

v: value (information to be extracted)

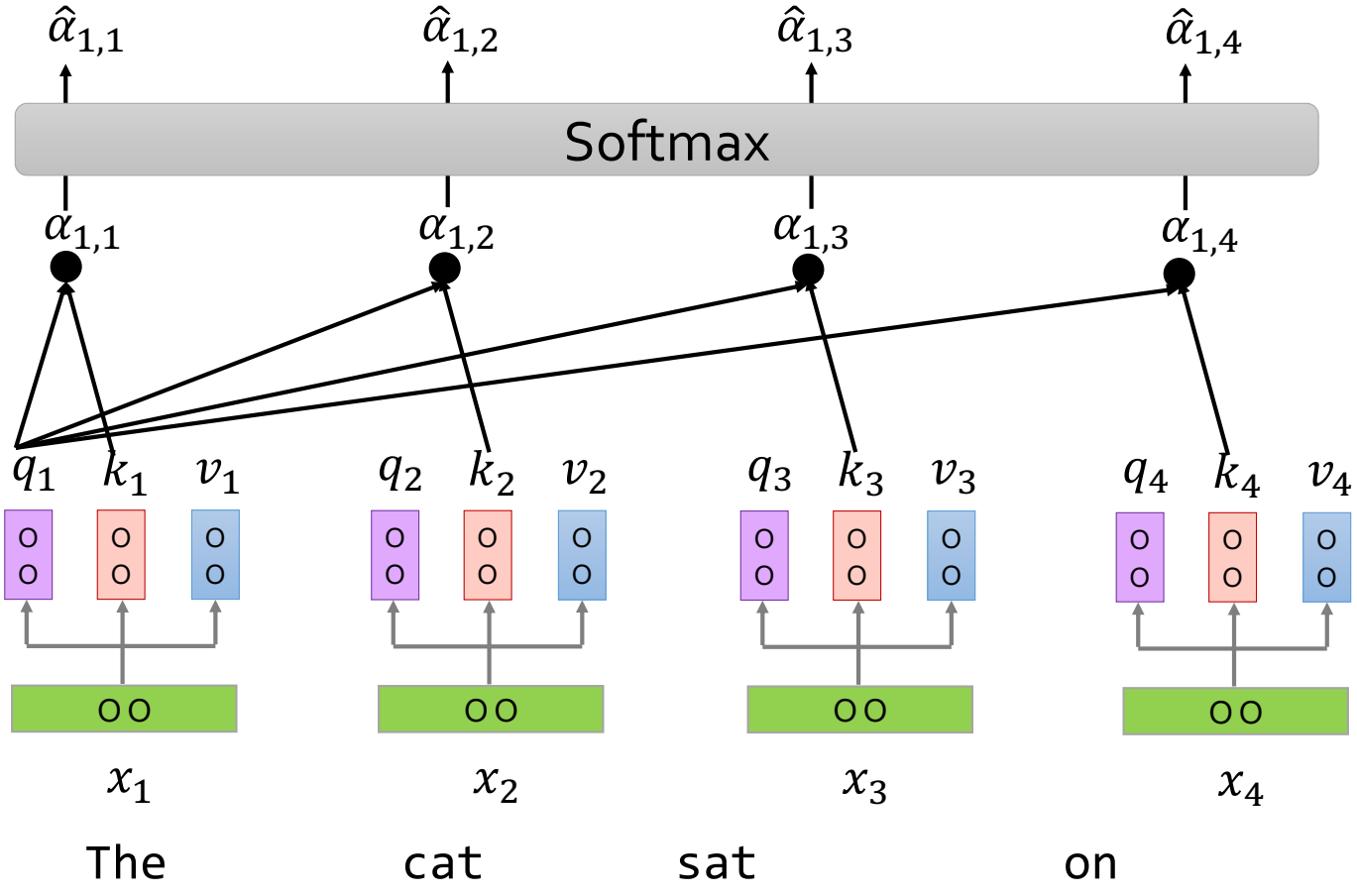
How much should "The" attend to other positions?

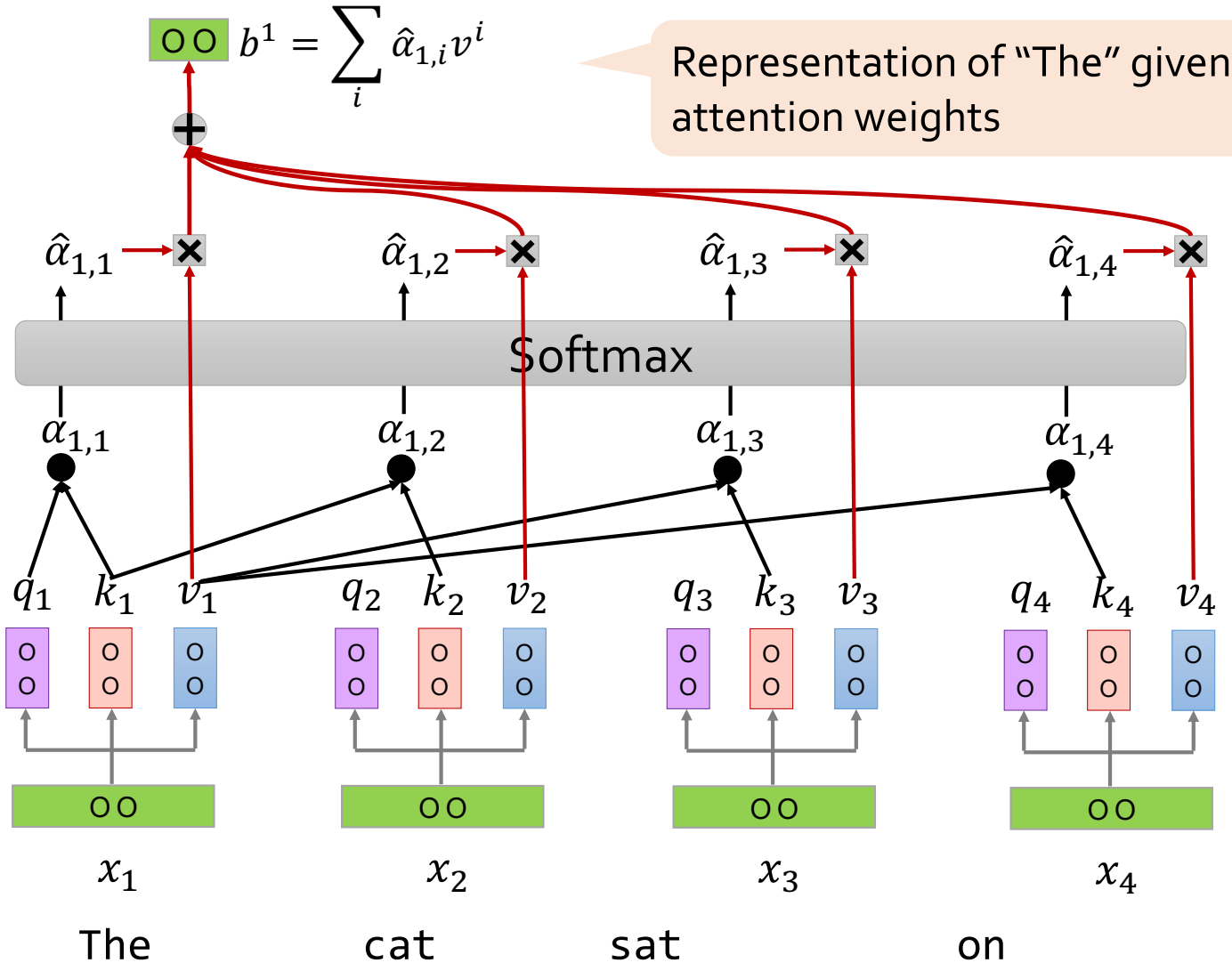




$$\sigma(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

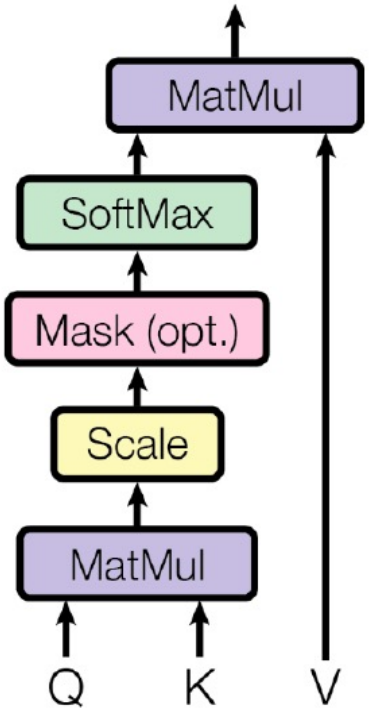
How much should "The" attend to other positions?




$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$
Representation of "The" given the attention weights
$\hat{\alpha}_{1,1}$
$\hat{\alpha}_{1,2}$
$\hat{\alpha}_{1,3}$
$\hat{\alpha}_{1,4}$
Softmax
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
$\alpha_{1,4}$
q_1 k_1 v_1
q_2 k_2 v_2
q_3 k_3 v_3
q_4 k_4 v_4
x_1
x_2
x_3
x_4
The
cat
sat
on

Transformer – Scaled dot product attention

- A query Q_i representing the token at position i attend to a sequence of tokens (represented by K): $Q_i K^T$.
- The result scaled by $\sqrt{d_k}$ (dimension of Q_i) is passed to a softmax function to compute a score for each of the tokens: $softmax(\frac{Q_i K^T}{\sqrt{d_k}})$
- A weighted sum of the Value vectors is calculated as the new representation of the token at position i: $softmax(\frac{Q_i K^T}{\sqrt{d_k}})V$
- Queries are stacked in one matrix Q, the output of the attention layer becomes: $softmax(\frac{QK^T}{\sqrt{d_k}})V$



Query - Key - Value

computations of vector representation of tokens “thinking” and “machines” IN CONTEXT via attention.

- W^Q, W^K, W^V are trainable matrices
- q,k,v vectors corresponding to the tokens in the context.
- For “thinking”: q1*k1 is the self similarity for this word which is normalized and softmaxed. The result multiplied by v_1.

- z1 = sum (q1, k1, v1) that represent “thinking” in the context of “thinking”).
- Similar process happens among “Thinining” and “Machines” resulting in the z2 vector that represents after all computations the vector that corresponds to “thinking” in the context of “machines”
- The W^* matrices represent attention weights among the tokens in the context.. ?

Zij representation of word i to the head j – then we MERGE and give them to the next layer.

- K, W^iV are intialised randomly and are trainable – i stand for the head
- X1 : [“Thinking” embedding: positional embedding of “Thinking”]
- q1 = X1 * W^Q, k1 = X1 * W^K, v1 = X1 * W^V
- qi*kj : attention among tokens i and j. Then normalized by softmax
- z1 = sum(v1j): representation of token 1 in context, j: words in context
- zi ‘s concatenated and passed to the next attention layers…

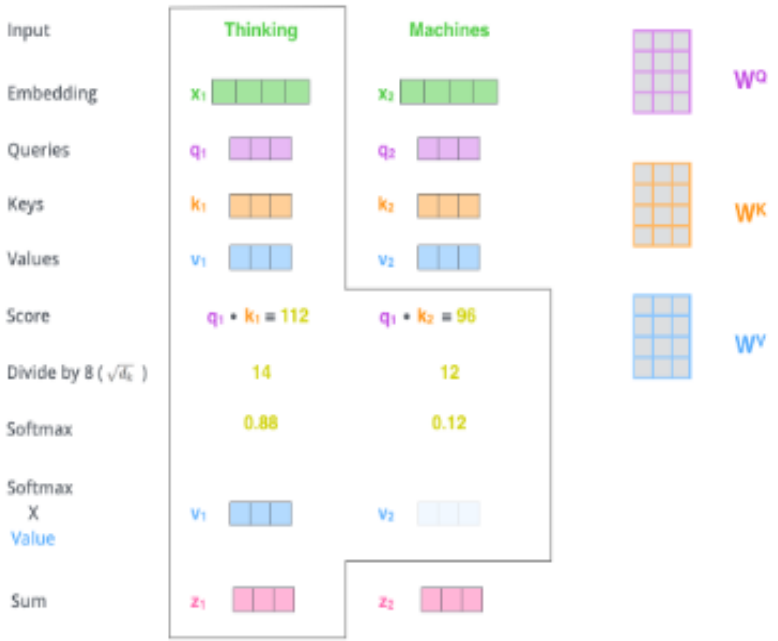
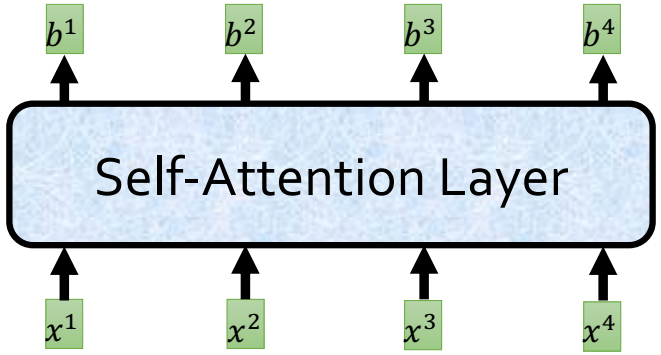


Figure: Self-attention calculation.

- Multi-Head Attention: $\{W_0^Q, W_0^K, W_0^V\}, \{W_1^Q, W_1^K, W_1^V\}, \ldots$

Recap: Self-Attention

- Attention is a powerful mechanism to create context-aware representations
- A way to focus on select parts of the input

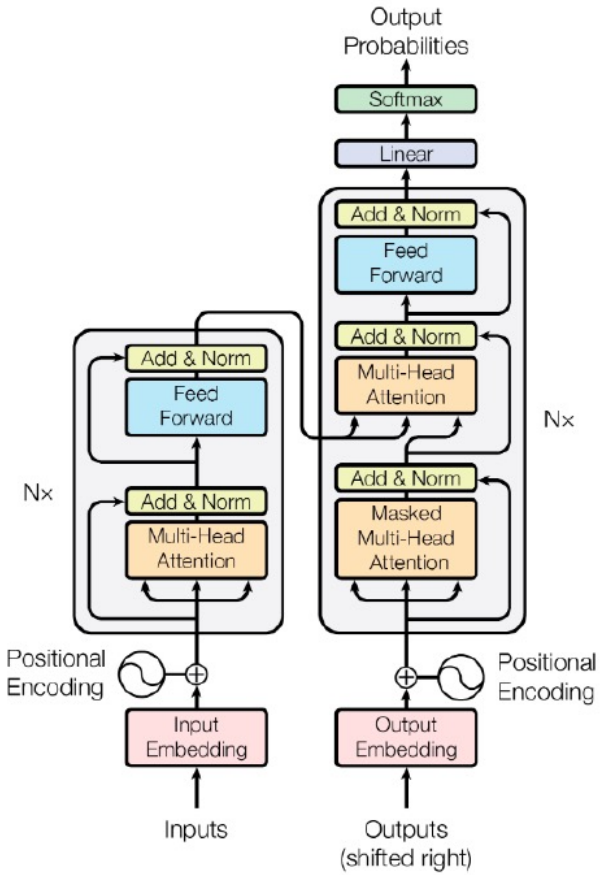


- Better at maintaining long-distance dependencies in the context.

[Attention Is All You Need, Vaswani et al. 2017]

Transformers- Attention

- **Self-attention:** Applied in the encoder and in the decoder:
  - In the encoder: Q, K and V come from the previous encoder sub-layer.
  - In the decoder: Q, K and V come from the previous decoder layer and a mask is applied to prevent the decoder from attending future positions.
- **Encoder-decoder attention:** Applied only in the decoder. Q come from the previous decoder layer. K and V come from the output representations of the encoder.



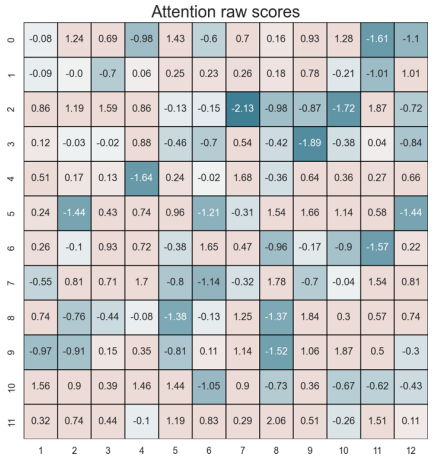
Self-Attention: Matrix Notation

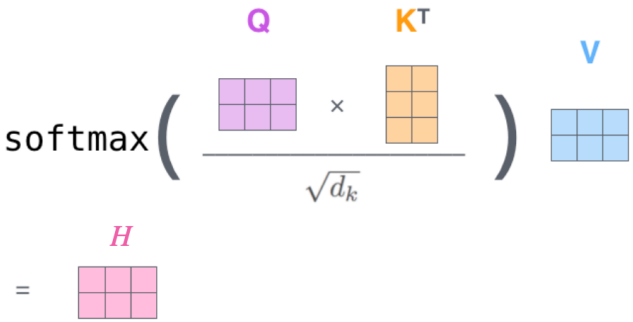
- Input sequence: $\mathbf{x} = [x_1, x_2, \ldots, x_n] \in \mathbb{R}^{n \times d}$
- Calculate:

$$\boldsymbol{Q} = \mathbf{x}\mathbf{W}^q \in \mathbb{R}^{n \times d}$$
$$\boldsymbol{K} = \mathbf{x}\mathbf{W}^k \in \mathbb{R}^{n \times d}$$
$$\boldsymbol{V} = \mathbf{x}\mathbf{W}^v \in \mathbb{R}^{n \times d}$$

- Pairwise similarity matrix between queries and keys: $\boldsymbol{Q}\boldsymbol{K}^T \in \mathbb{R}^{n \times n}$
- Attention output:

$$\text{Attention}(\mathbf{x}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d}}\right)\boldsymbol{V} \in \mathbb{R}^{n \times d}$$

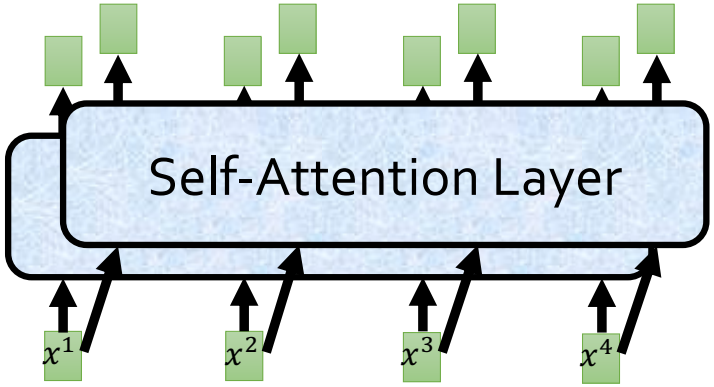




Multi-Headed Self-Attention



- **Issue:** Each head has limited capacity, e.g., it cannot attend to too many things at the same time.
- **Main idea:** Allows model to jointly attend to information from different representation subspaces (like ensembling).
- Multiple parallel attention layers.
  - Each attention layer has its own parameters.
  - Concatenate the results and run them through a linear projection.



[Attention Is All You Need, Vaswani et al. 2017]

Multi-Headed Self-Attention

- Just concatenate all the heads and apply an output projection.

$$\text{head}_i = \text{Attention}\left(\mathbf{x}\mathbf{W}_i^q, \mathbf{x}\mathbf{W}_i^k, \mathbf{x}\mathbf{W}_i^v\right)$$
$$\text{MultiHeadedAttention}(\mathbf{x}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\boldsymbol{W}^O$$

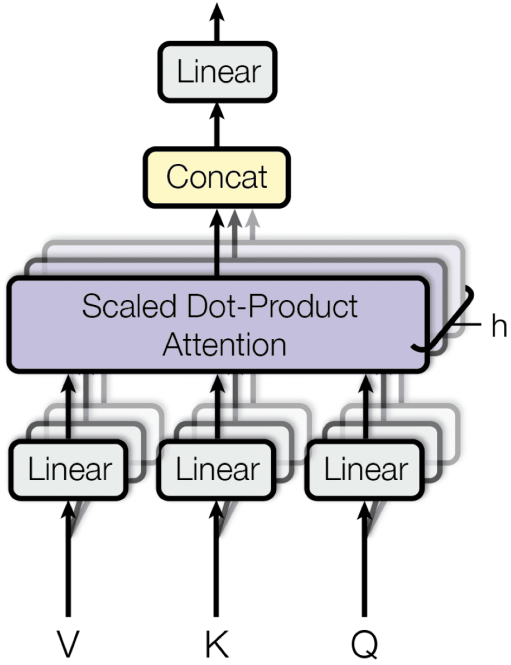
- Previously, used the following dimensions for **single**-head SA:

$$\mathbf{W}_i^q \in \mathbb{R}^{d\times d}, \quad \mathbf{W}_i^k \in \mathbb{R}^{d\times d}, \quad \mathbf{W}_i^v \in \mathbb{R}^{d\times d},$$

- In practice, use a reduced dimension for each head.

$$\mathbf{W}_i^q \in \mathbb{R}^{d\times\frac{d}{h}}, \quad \mathbf{W}_i^k \in \mathbb{R}^{d\times\frac{d}{h}}, \quad \mathbf{W}_i^v \in \mathbb{R}^{d\times\frac{d}{h}}, \quad \boldsymbol{W}^O \in \mathbb{R}^{d\times d}$$

- The total computational cost is similar to that of single-head attention with full dimensionality.



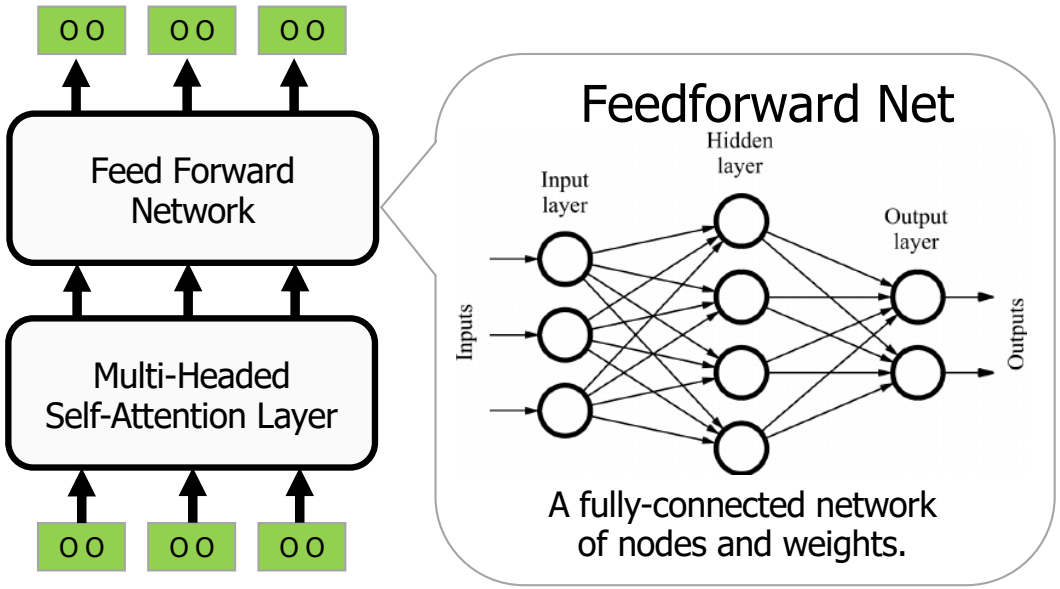
h: number of heads
d: feature dimension
in output of SA

Combine with FFN

- Add a feed-forward network to add more expressivity.
  - This applies another nonlinearity to the representations (or "post-process" them).

$$\text{FFN}(\mathbf{x}) = f(\mathbf{x}\boldsymbol{W}_1 + b_1)\boldsymbol{W}_2 + \boldsymbol{b}_2$$
$$\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_{\text{ff}}},$$
$$\boldsymbol{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$$

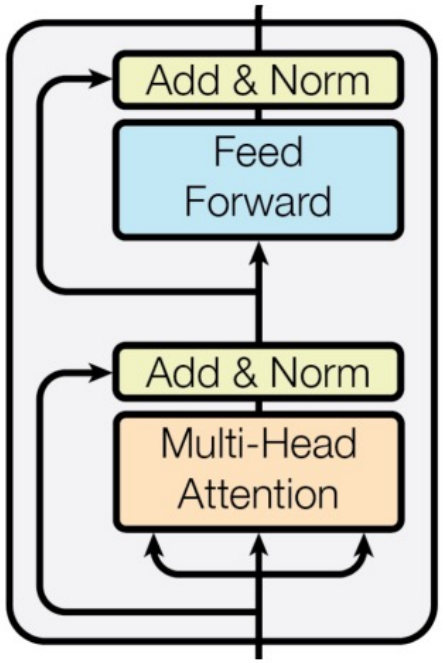
  - Usually, the dimensionality of the hidden feedforward layer d_{ff} is 2-8 times larger than the input dimension d.



How Do We Prevent Vanishing Gradients?

- Residual connections let the model "skip" layers
  - These connections are particularly useful for training deep networks

- Use layer normalization to stabilize the network and allow for proper gradient flow



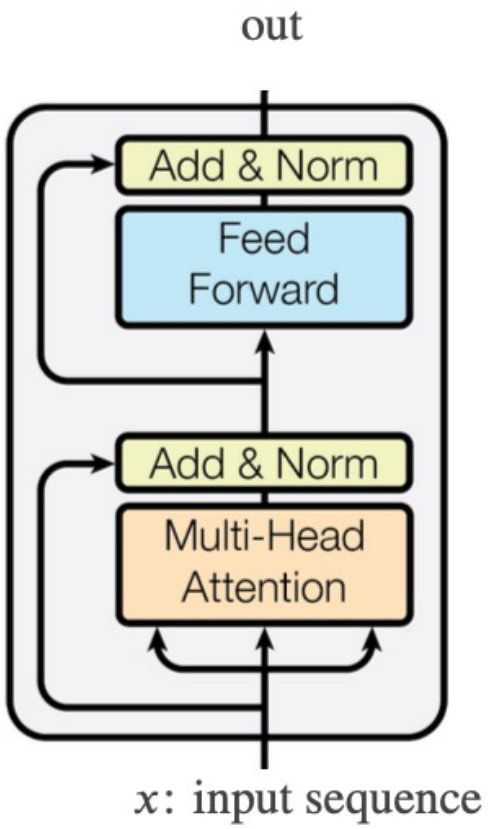
[Attention Is All You Need, Vaswani et al. 2017]

Putting it Together: Self-Attention Block

Given input $\mathbf{x}$:

$$\tilde{\mathbf{x}} = \text{MultiHeadedAttention}(\mathbf{x}; \mathbf{W}^q, \mathbf{W}^k, \mathbf{W}^v)$$
$$\mathbf{x} = LayerNorm\ (\tilde{\mathbf{x}} + \boldsymbol{x})$$

$$\tilde{\mathbf{x}} = \text{FFN}(\mathbf{x}) = f(\mathbf{x}W_1 + b_1)W_2 + b_2$$
$$\text{out} = LayerNorm(\tilde{\mathbf{x}} + \mathbf{x})$$

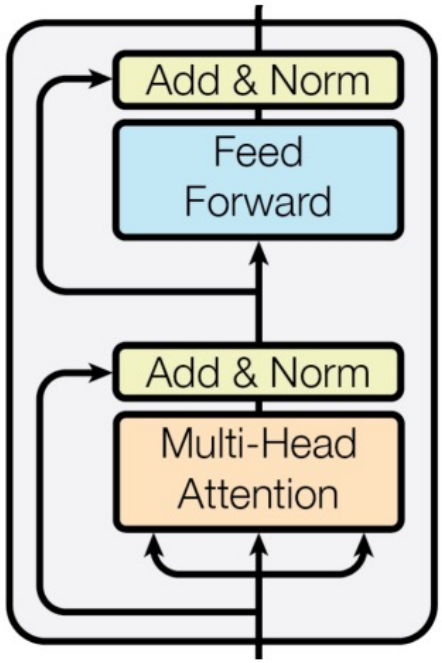


[Attention Is All You Need, Vaswani et al. 2017]

Summary: Self-Attention Block

- **Self-Attention:** A critical building block of modern language models.
  - The idea is to compose meanings of words weighted according some similarity notion.

- **Next:** We will combine self-attention blocks to build various architectures known as Transformer.



The Bottleneck in Autoregressive Generation

- **The Challenge: Redundant Computation**
- **Context:** Autoregressive generation in LLMs generates text one token at a time, utilizing all previously generated words as context.
- **Inefficiency:** In a naive implementation, the model receives a new token at step *t* and recomputes the Key (K) and Value (V) matrices for **all** [1 .. *t*] every single time.
- **Visualizing the Waste:**
  - *Step 1:* Compute attention for token 1.
  - *Step 2:* Recompute K and V for token 1, plus the new ones for token 2.
  - *Step 3:* Recompute K and V for tokens 1 and 2, plus the new ones for token 3.
- **Conclusion:** massive repetition of computation that has already been done.

The Core Solution: KV Caching

- **Mechanism: Store Instead of Recompute**
- **Concept:** solve the redundancy problem, *store the computed K and V vectors from previous steps* rather than discarding them.
- **The Process at Step *t***
  - **Retrieve:** Access the existing cache containing [K_1,...K_{t-1}] and [$V1_1$,...V_{t-1}]
  - **Compute:** Calculate **only** the new $K_t$and V_t for the current token.
  - **Concatenate:** Combine cached vectors with the new vectors to form K_{new} and V_{new}.
- **Outcome:** The attention computation

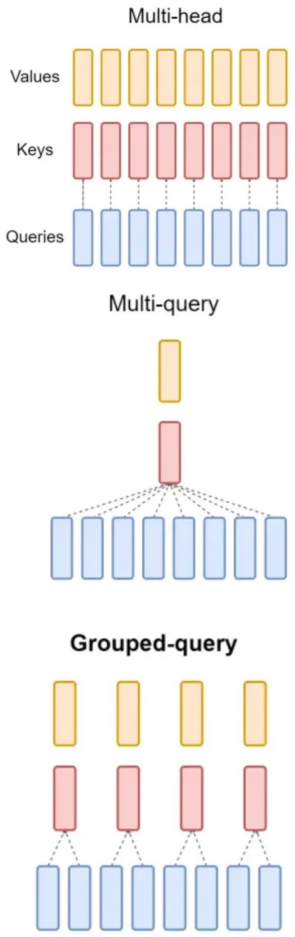
$$\text{Output}_t = \text{softmax}(\frac{Q_t K_{new}^T}{\sqrt{d_k}})V_{new}$$

utilizes all previous context without the computational cost of re-processing it.

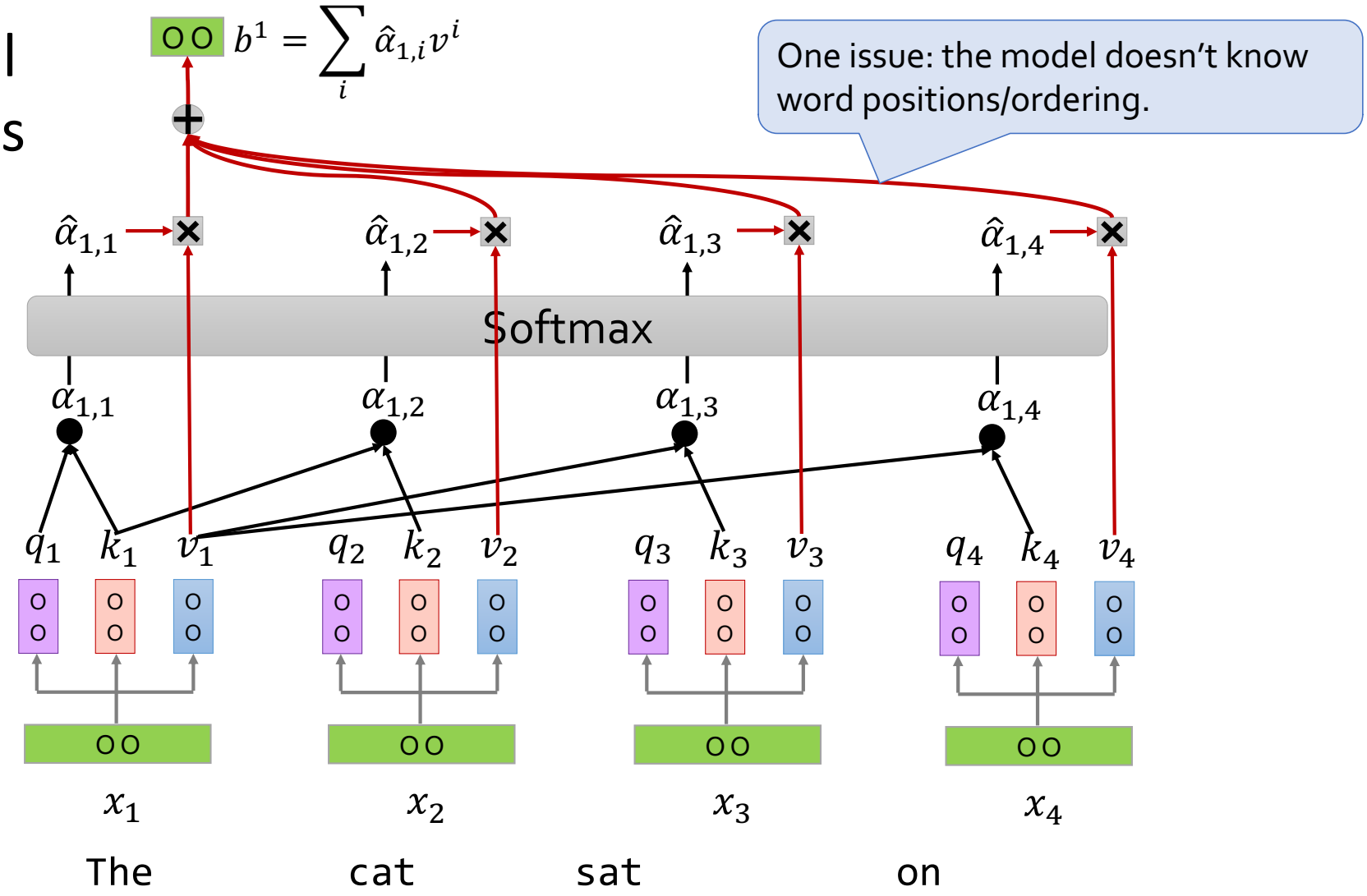
Interesting visualization at: https://huggingface.co/blog/not-lain/kv-caching

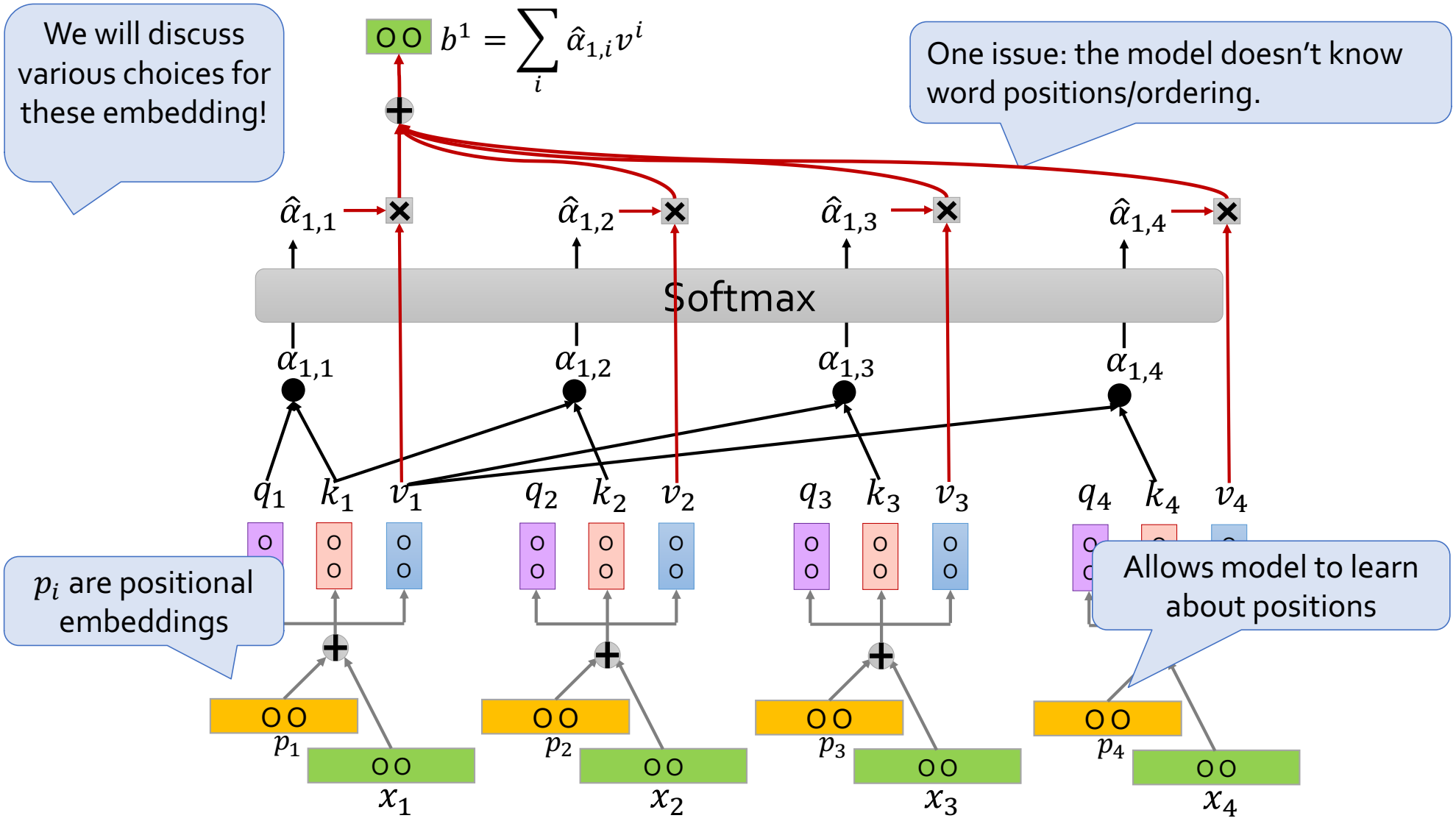
Variants of Attention (MHA, MQA, GQA)

- **Optimizing for Speed and Memory**
- **Multihead Attention (MHA):**
  - *Structure:* Every attention head maintains its **own** Key and Value.
  - *Performance:* Result is high quality but slower, showing a high, constant time per sample in performance benchmarks.
- **Multi-Query Attention (MQA):**
  - *Structure:* Uses **one shared** Key and Value for *all* attention heads.
  - *Performance:* Makes decoding significantly faster - only a small drop in quality.
- **Grouped Query Attention (GQA):**
  - *Structure:* A middle ground that groups heads together to share Keys and Values **within each group**.
  - *Benefit:* improves MQA reducing training instability - maintaining good speed.
- **Summary:** As number of groups decreases (moving toward MQA), time per sample drops significantly compared to standard MHA.



Positional encodings




We will discuss various choices for these embedding!
$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$
One issue: the model doesn't know word positions/ordering.
$\hat{\alpha}_{1,1}$
$\hat{\alpha}_{1,2}$
$\hat{\alpha}_{1,3}$
$\hat{\alpha}_{1,4}$
Softmax
$\alpha_{1,1}$
$\alpha_{1,2}$
$\alpha_{1,3}$
$\alpha_{1,4}$
q_1 k_1 v_1
q_2 k_2 v_2
q_3 k_3 v_3
q_4 k_4 v_4
p_i are positional embeddings
Allows model to learn about positions
p_1
p_2
p_3
p_4
x_1
x_2
x_3
x_4

Absolute Positional Embeddings

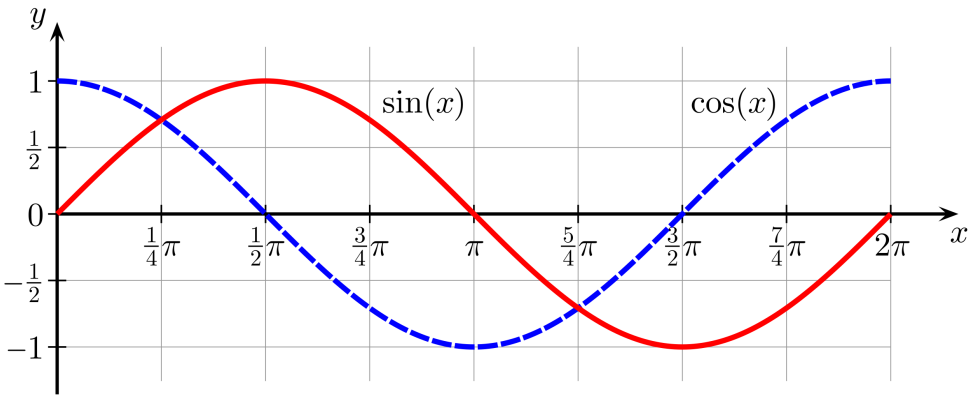
- The idea is to create vectors that uniquely encoder each position.
- For example, consider vectors of binary values.
  - Example below shows 4-dimensional position encodings for 16 positions.

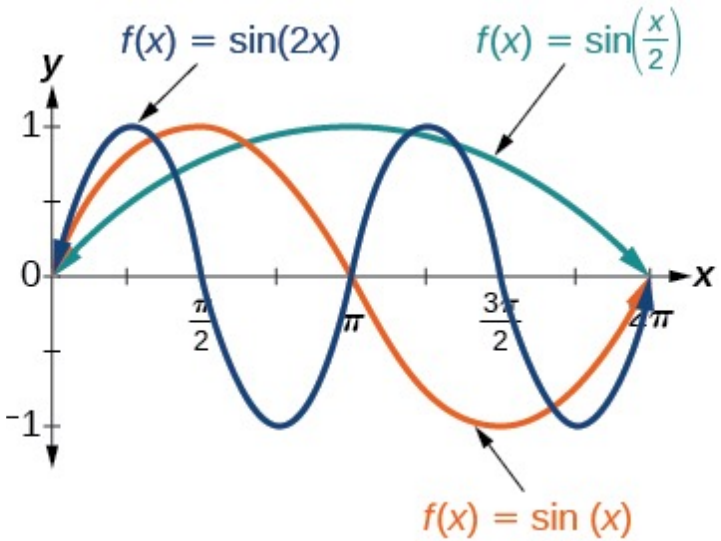
0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

Issues: Discrete numbers; the information is localized around a few bits; embedding depends on the number of positions ☹

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Math Recap: Sine and Cosine Functions





Absolute Positional Embeddings

- Let t be a desired position. Then the i-th element of the positional vector is:

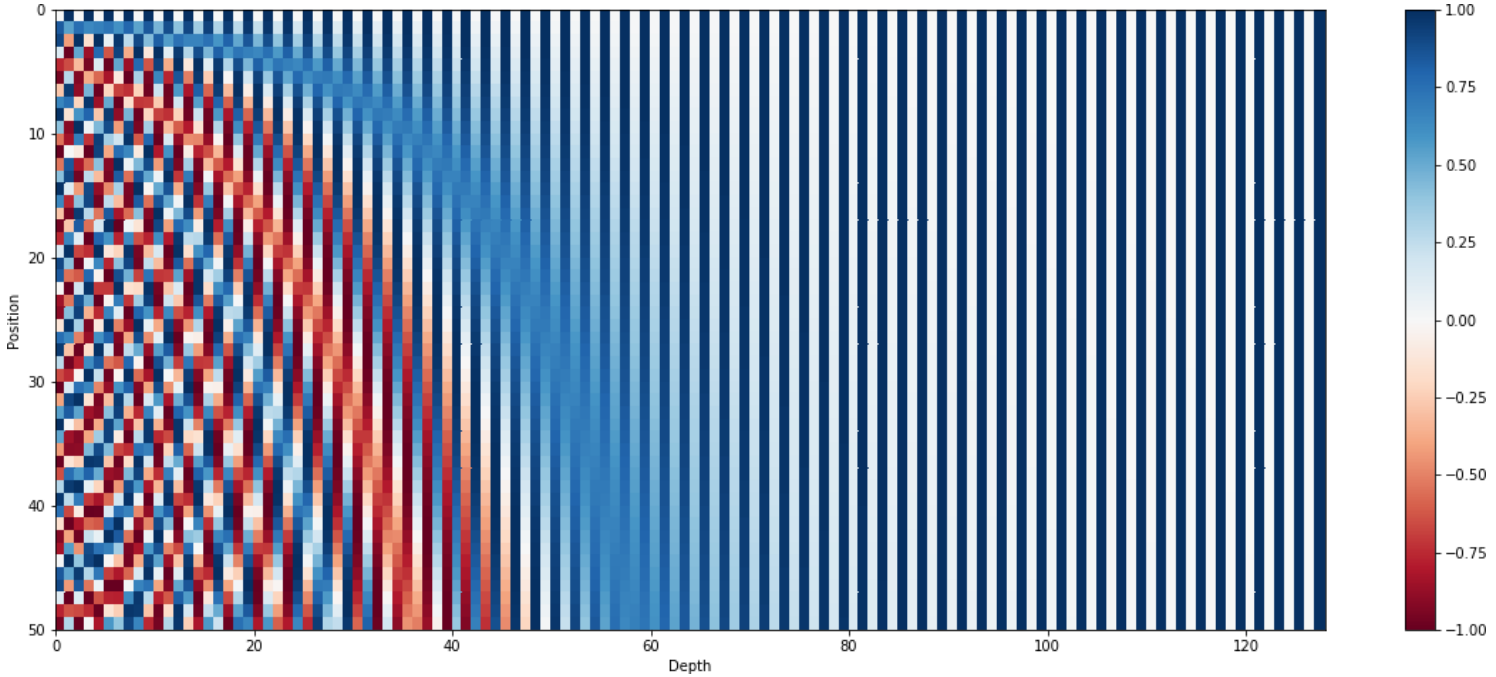
$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases} \qquad \omega_k = \frac{1}{10000^{2k/d}}$$

- Here d is the maximum dimension of each positional vector.
- This provides unique vectors for each position.

https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Visualizing Absolute Positional Embeddings

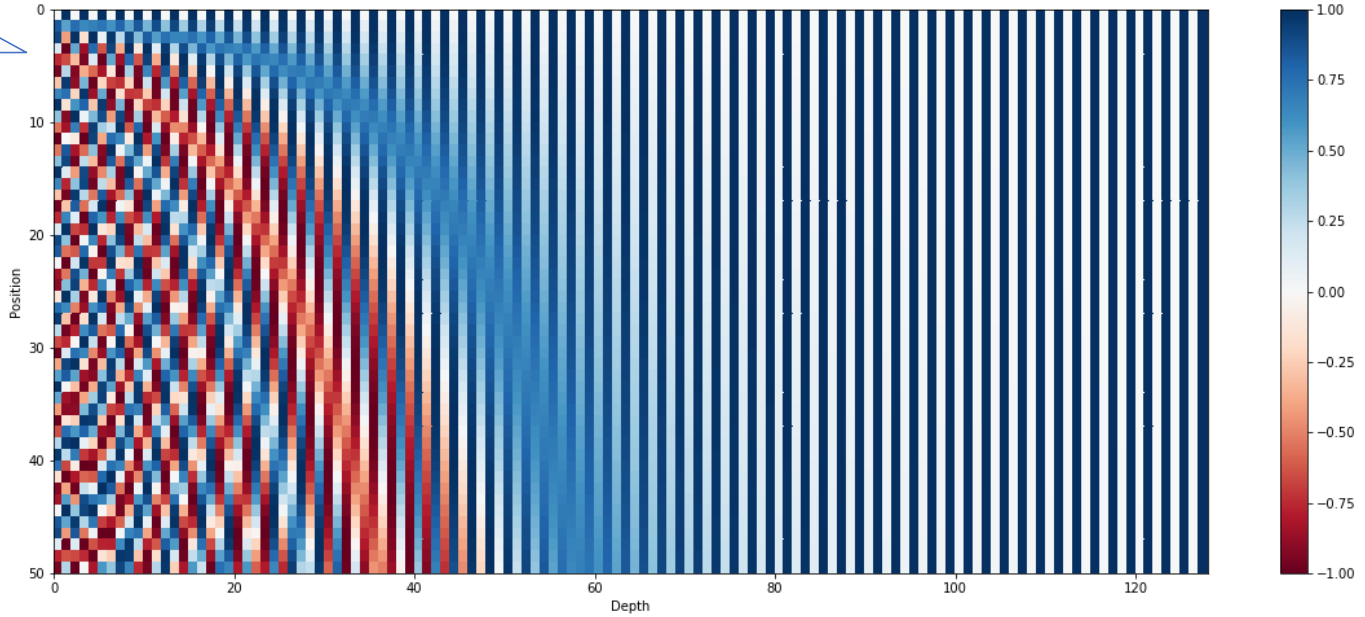
- Here positions range from 0-50, for an embedding dimension of 130.



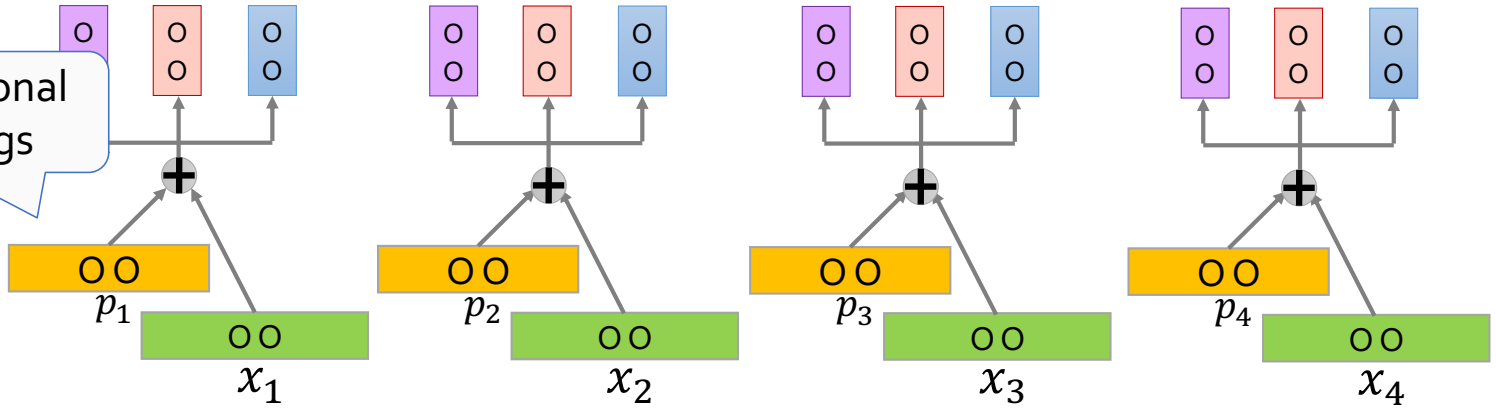
Notice, **any pair of rows are different!**

Sine/Cosine encoding

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



p_i are positional embeddings



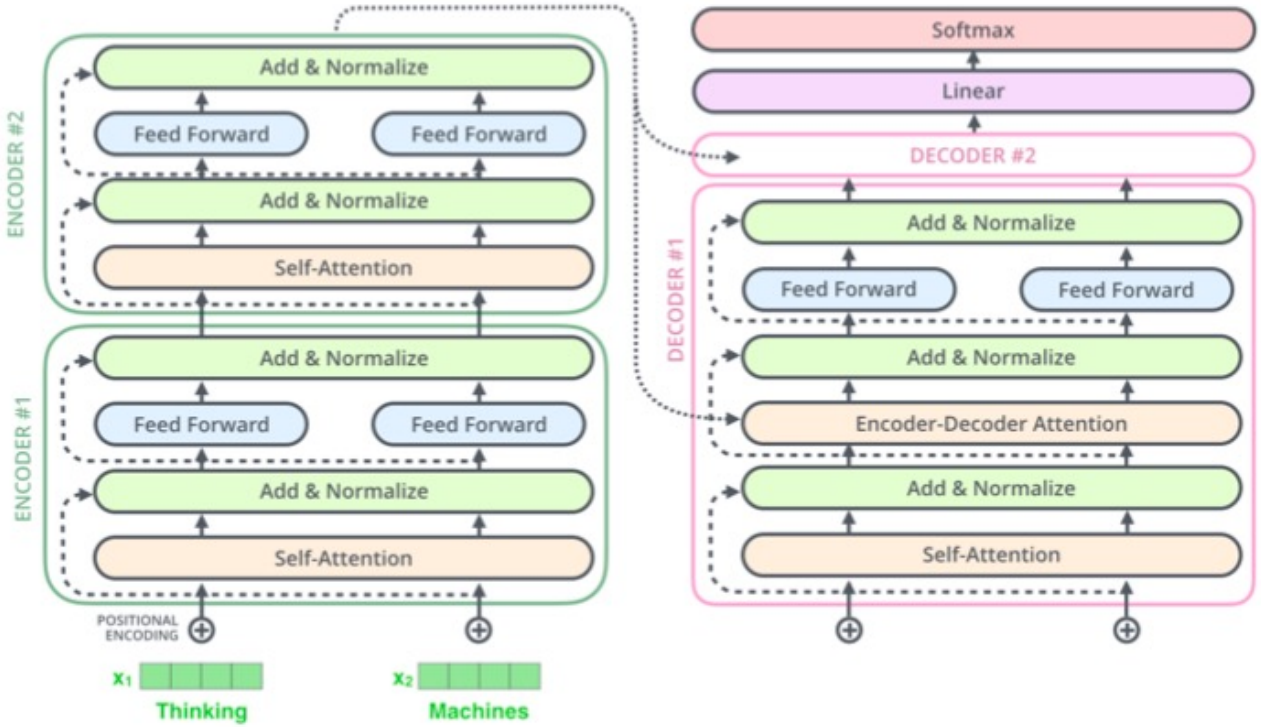
Recap— positional encoding

- They inform Transformer about word orderings.

- The original proposal: fixed sine/cosine embeddings added to word embeddings.
  - While this is a reasonable start, later we will see more sophisticated alternatives.

- Next: let's think about running transformer on language data.

Tranfsormer Architecture



1) https://jalammar.github.io/illustrated-transformer/
2) http://nlp.seas.harvard.edu/annotated-transformer/

Pretraining Large Language Models

- Pre-training objectives:
  - Masked LM, Causal LM, PrefixLM, denoising
- General purpose: modelling language + knowledge
- Encoder Only (Bert)
- Encoder Decoder (T5)
- Decoder Only (GPT)

Language Model in LLMs…

Models that assign probabilities to sequences of words are called **language models**

- *P*(all of a sudden I notice three guys standing on the sidewalk) > *P*(on guys all I of notice sidewalk three a sudden standing the)
- *P*(?|Please turn your homework ?) ⇒ in > over > > the > elephant *uni-directional*
  - shorter context size, less accurate prediction: *P*(?|your homework ?)
- *P*(?|Please turn ? homework) *bi-directional*
- *P*(he briefed reporters on the main contents of the statement | il a informé les journalistes des principaux éléments de la déclaration) >

  P(he to reporters introduced main content | il a informé les journalistes des principaux éléments de la déclaration)

  sequence-to-sequence

1) https://web.stanford.edu/~jurafsky/slp3/

BERT |Decoder Only | MLM

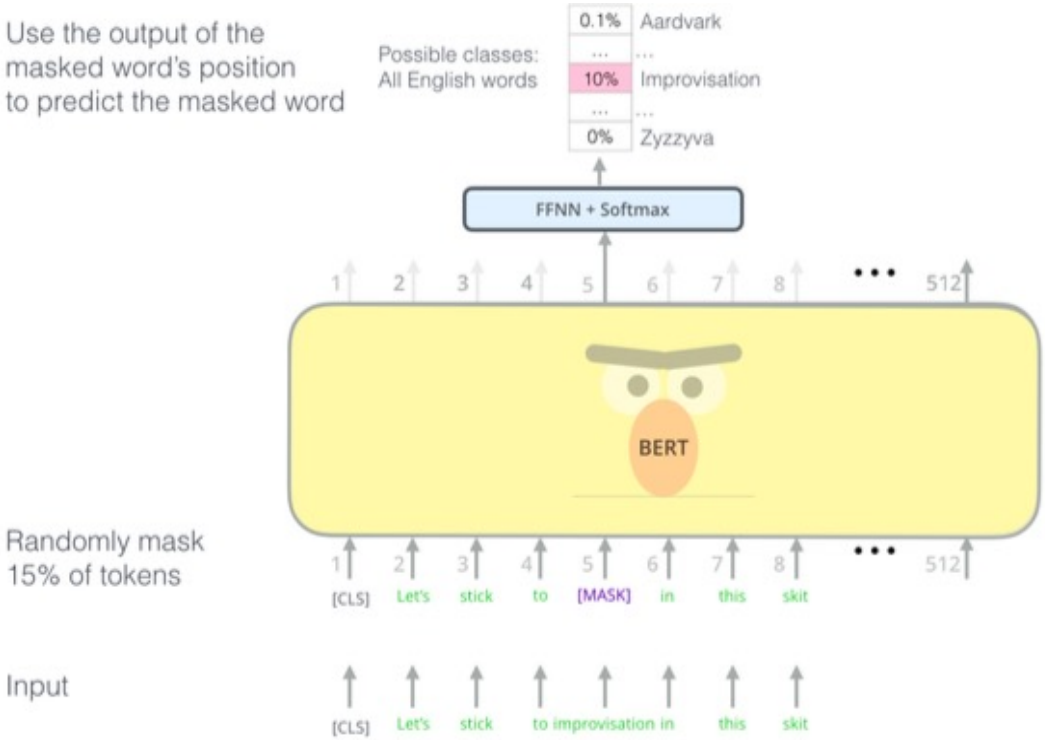
- Masked Language Modeling (MLM) predicts a masked token in a sequence, and the model can attend to tokens bidirectionally [1].
- `*Distributional hypothesis* of language?
  - Word in a context are semantically similar

- Contextualized word embeddings:

  Apple presented the new iphone X

  vs

  She picked a ripe apple from the tree



[1] http://jalammar.github.io/illustrated-bert/

Sentence Bert

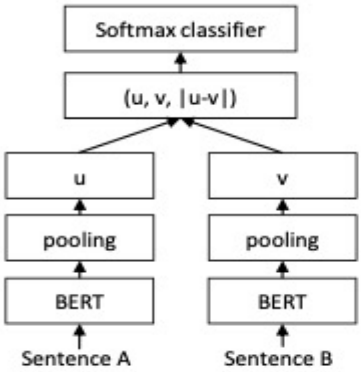


Figure: SBERT architecture with classification objective function, e.g., for fine-tuning on SNLI dataset. The two BERT networks have tied weights (siamese network structure).

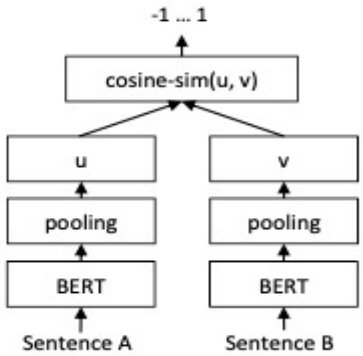


Figure: SBERT architecture at inference, for example, to compute similarity scores. This architecture is also used with the regression objective function.

- Semantic textual similarity: `https://www.sbert.net`

```
model = SentenceTransformer("all-MiniLM-L6-v2")
emb1 = model.encode("This is a red cat with a hat.")
emb2 = model.encode("Have you seen my red cat?")
cos_sim = util.cos_sim(emb1, emb2)
```

- Keyword extraction: `https://github.com/MaartenGr/KeyBERT`

1) Reimers, Nils, and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks." arXiv preprint arXiv:1908.10084 (2019).

Improving Language Understanding by Generative Pre-Training (GPT)

GPT – Unsupervised pretraining

- Data:
  - BooksCorpus dataset:
    - over 7,000 unique unpublished books from a variety of genres including Adventure, Fantasy, and Romance.
    - contains long stretches of contiguous text, allows the generative model to learn to condition on long-range information.
  - 1B Word Benchmark used by ELMo - approximately the same size

Improving language understanding by generative pre-training, A Radford, K Narasimhan, T Salimans, I Sutskever, https://www.mikecaptain.com/resources/pdf/GPT-1.pdf

GPT, Decoder-only, CLM

Causal language modeling (CLM) predicts the next token in a sequence of tokens, and the model can only attend to tokens on the left.

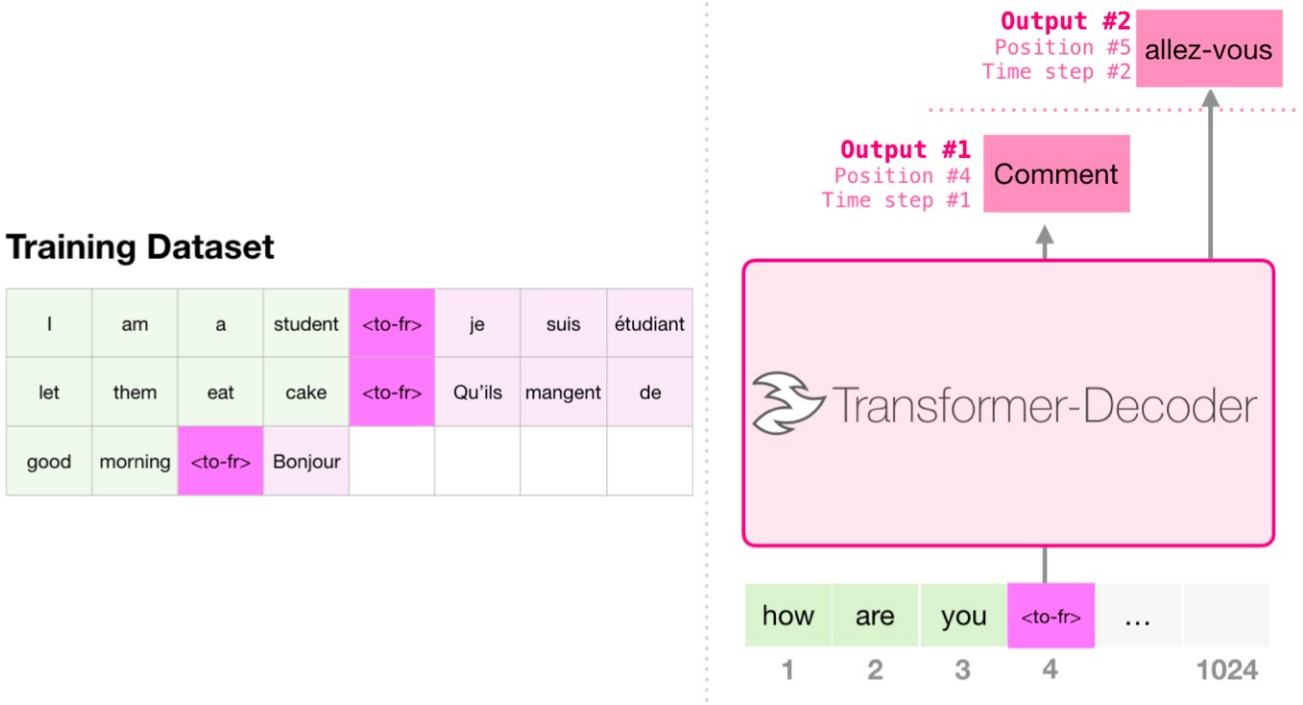


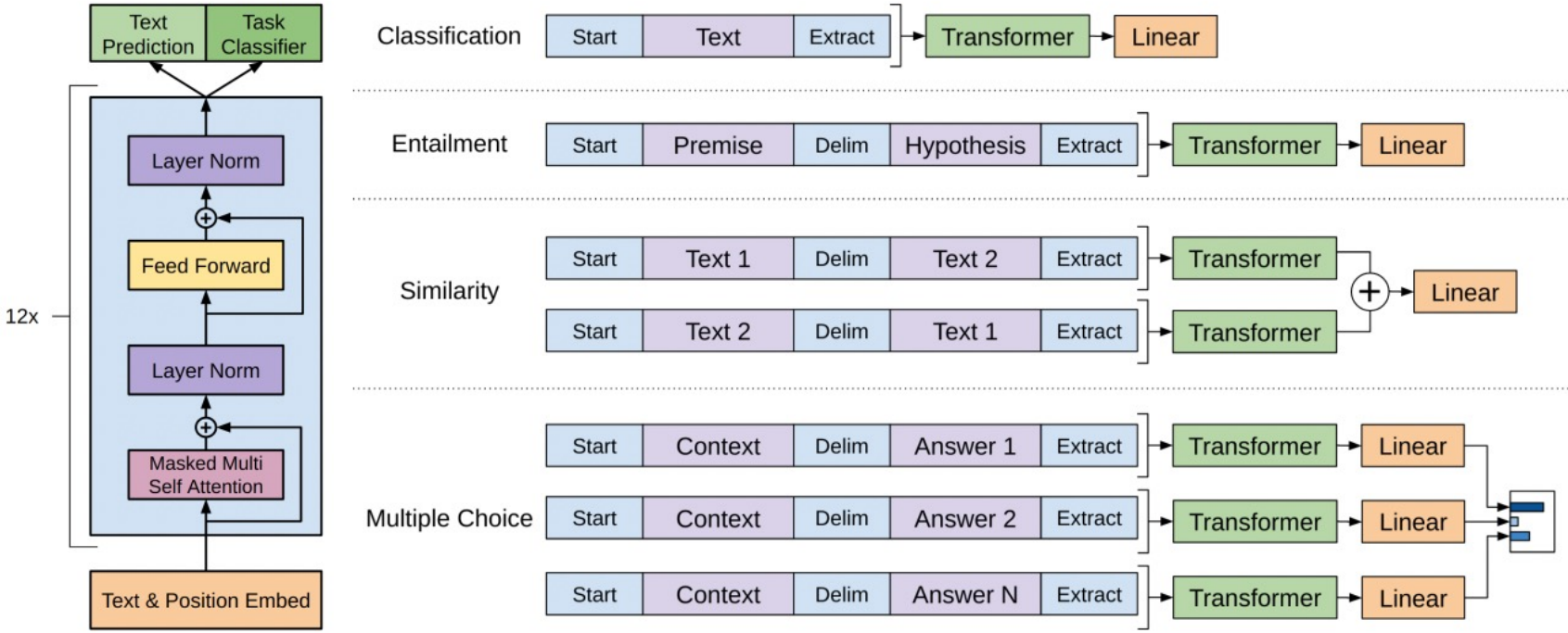
Figure: GPT: **outputs one token at a time.**[1] <to-fr> can be generalised into natural language instructions.

[1] https://jalammar.github.io/illustrated-gpt2/

GPT – model specifications

- largely follows the original transformer work
- *12-layer decoder-only transformer* with masked self-attention heads (768 dimensional states and 12 attention heads).
- train for 100 epochs on minibatches of 64 randomly sampled, contiguous sequences of 512 tokens.
- Regularization: attention dropouts with a rate of 0.1
- position embeddings learned instead of the sinusoidal proposed in original work.

GPT – Supervised fine tuning



GPT – Supervised finetuning

- Natural Language Inference (entailement)

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

- Q&A

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	77.6	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	60.2	50.3	53.3
Finetuned Transformer LM (ours)	**86.5**	**62.9**	**57.4**	**59.0**

- Semantic similarity and classification

Method	**Classification**		**Semantic Similarity**			**GLUE**
	CoLA (mc)	SST2 (acc)	MRPC (F1)	STSB (pc)	QQP (F1)	
Sparse byte mLSTM [16]	-	**93.2**	-	-	-	-
TF-KLD [23]	-	-	**86.0**	-	-	-
ECNU (mixed ensemble) [60]	-	-	-	81.0	-	-
Single-task BiLSTM + ELMo + Attn [64]	35.0	90.2	80.2	55.5	66.1	64.8
Multi-task BiLSTM + ELMo + Attn [64]	18.9	91.6	83.5	72.8	63.3	68.9
Finetuned Transformer LM (ours)	**45.4**	91.3	82.3	**82.0**	**70.3**	**72.8**

T5, Encoder-decoder, MLM

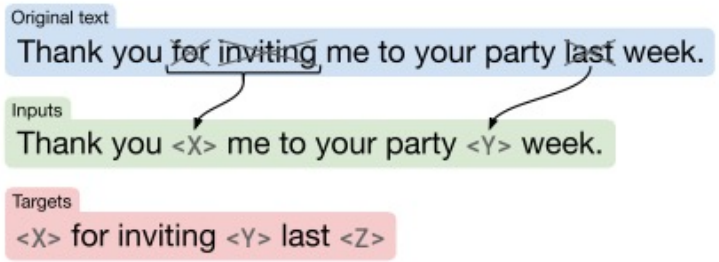


Figure: **PRE-TRAINING**: The words "for", "inviting" and "last" (marked with an ×) are randomly chosen for corruption. Each consecutive span of corrupted tokens is replaced by a sentinel token (shown as <X> and <Y>) that is unique over the example. The output sequence then consists of the dropped-out spans, delimited by the sentinel tokens used to replace them in the input plus a final sentinel token <Z>.

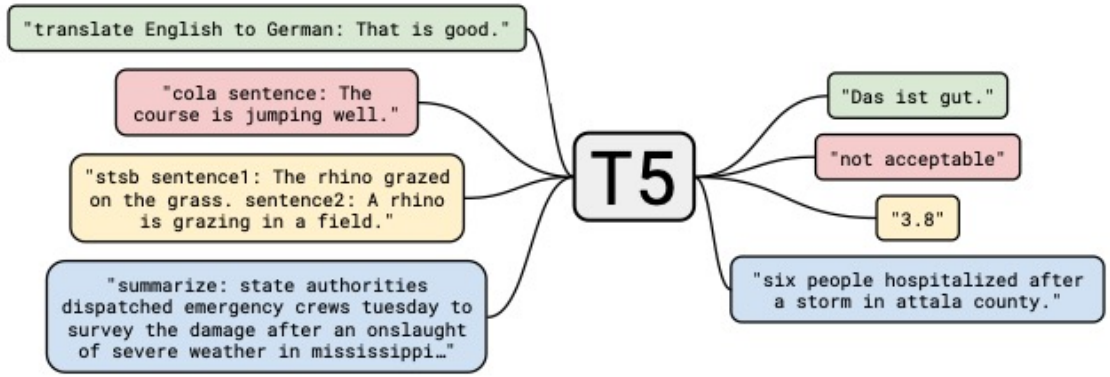
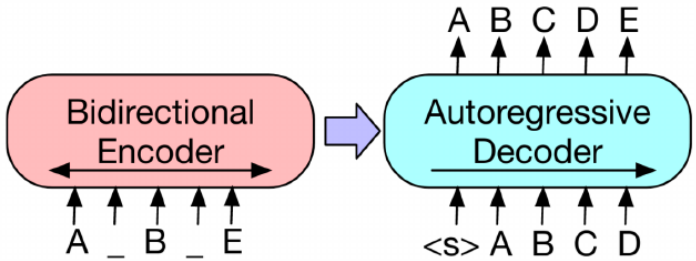


Figure: **INSTRUCTION-TUNING**: every task is cast as feeding text as input to generate some target text.[1]

1) Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." The Journal of Machine Learning Research 21.1 (2020): 5485-5551.

Bart Overview

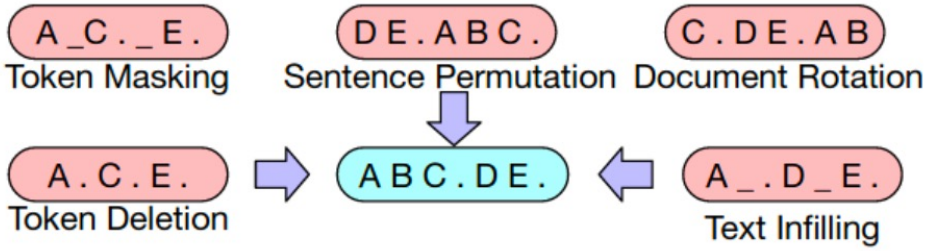


- A denoising autoencoder, that uses a bidirectional encoder and a left-to-right autoregressive decoder.
- Uses the standard Transformer architecture, with 12 encoder layers and 12 decoder layers.
- Pre-trained on 160GB of english text.
- Given its architecture, Bart can be used in both discriminative and generative tasks.

BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

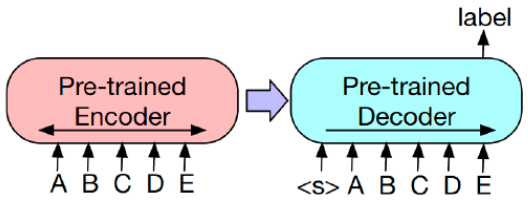
Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, Luke Zettlemoyer

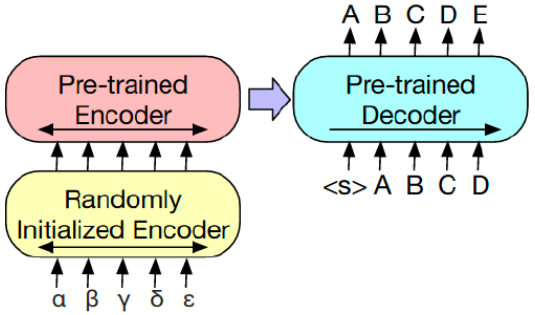
Bart – Pre-training Task



- The Pre-training task is to denoise a corrupted text. Several noising functions are tested:
  - Token Masking: random tokens are replaced with a [MASK] token.
  - Token Deletion: random tokens are deleted.
  - Text Infilling: Random text spans are sampled with span length drawn from Poisson distribution ($\lambda = 3$) and replaced with [MASK] token.
  - Sentence Permutation: Sentences are shuffled.
  - Sentence Rotation: sentences are rotated.
- Ablation experiments show that a combination of text infilling and sentence permutation perform consistently good on supervised tasks.

Bart – Fine tuning





- Bart can be fine-tuned on different types of tasks:
  - **Sequence Generation:** Given the decoder is autoregressive, Bart can be directly used to generate sequences in the same language as the input sequence.
  - **Sequence classification:** The final hidden state of the final decoder token is used. (Figure to the left)
  - **Tokens classification:** The top hidden state of the each decoder token is used.
  - **Machine translation:** The encoder embedding layer is replaced with an encoder that encodes the source language. (Figure to the right)

Bart – Evaluation

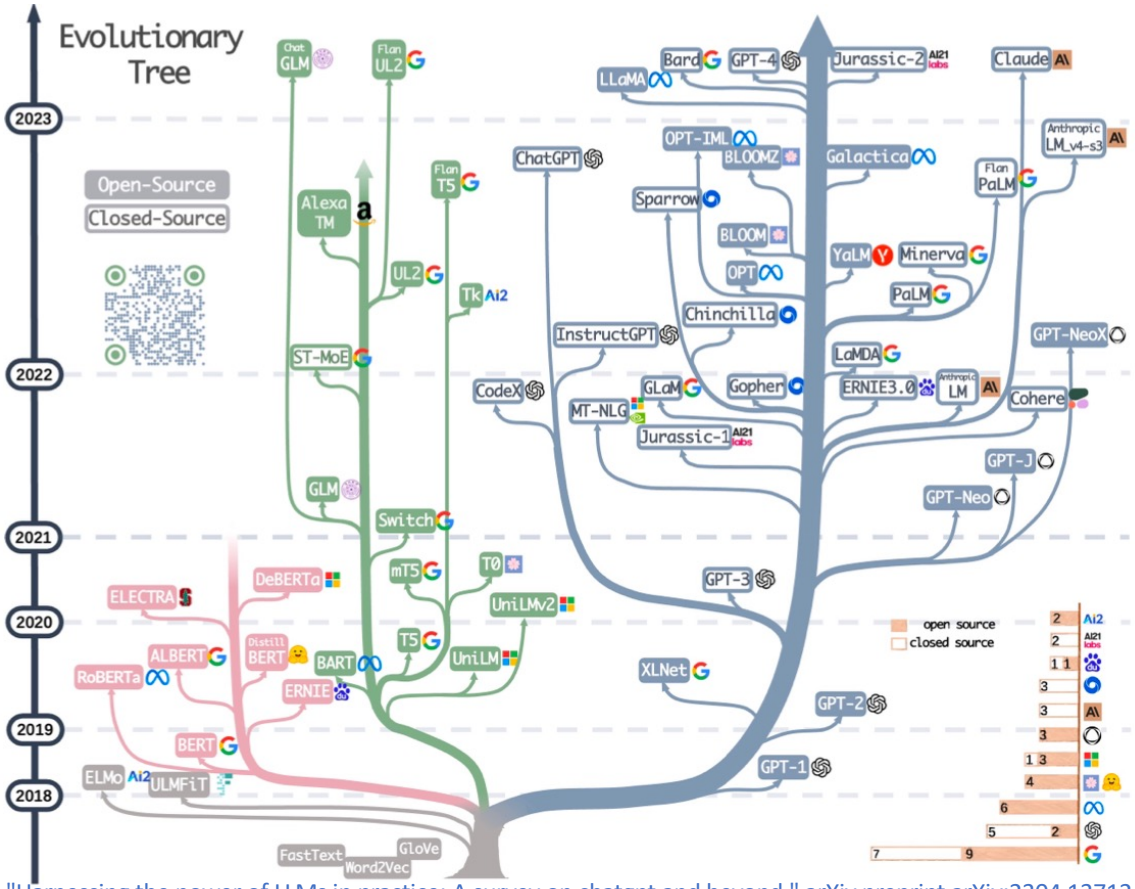
	SQuAD 1.1 EM/F1	SQuAD 2.0 EM/F1	MNLI m/mm	SST Acc	QQP Acc	QNLI Acc	STS-B Acc	RTE Acc	MRPC Acc	CoLA Mcc
BERT	84.1/90.9	79.0/81.8	86.6/-	93.2	91.3	92.3	90.0	70.4	88.0	60.6
UniLM	-/-	80.5/83.4	87.0/85.9	94.5	-	92.7	-	70.9	-	61.1
XLNet	**89.0**/94.5	86.1/88.8	89.8/-	95.6	91.8	93.9	91.8	83.8	89.2	63.6
RoBERTa	88.9/**94.6**	**86.5/89.4**	**90.2/90.2**	96.4	92.2	94.7	**92.4**	86.6	**90.9**	**68.0**
BART	88.8/**94.6**	86.1/89.2	89.9/90.1	**96.6**	**92.5**	**94.9**	91.2	**87.0**	90.4	62.8

	CNN/DailyMail			XSum		
	R1	R2	RL	R1	R2	RL
Lead-3	40.42	17.62	36.67	16.30	1.60	11.95
PTGEN (See et al., 2017)	36.44	15.66	33.42	29.70	9.21	23.24
PTGEN+COV (See et al., 2017)	39.53	17.28	36.38	28.10	8.02	21.72
UniLM	43.33	20.21	40.51	-	-	-
BERTSUMABS (Liu & Lapata, 2019)	41.72	19.39	38.76	38.76	16.33	31.15
BERTSUMEXTABS (Liu & Lapata, 2019)	42.13	19.60	39.18	38.81	16.50	31.27
BART	**44.16**	**21.28**	**40.90**	**45.14**	**22.27**	**37.25**

mBart – Multilingual Bart

- Multilingual sequence-to-sequence denoising auto-encoder.
- Follows BART architecture and objectives, but trained on several mono-language corpora across several languages.
- Can be directly fine-tuned on machine translation supervised tasks.
- Shows important performance gain in low-resource settings.
- mBART25 pre-trained model is published: trained on 1369.6 GB of raw text data from 25 different languages corpora.

Evolutionary tree of LLMs



1) Yang, Jingfeng, et al. "Harnessing the power of LLMs in practice: A survey on chatgpt and beyond." arXiv preprint arXiv:2304.13712 (2023).

Scaling laws for LLMs

Scaling up the amount of data D, compute power C, and model parameter $N \Rightarrow$ **predictable smooth continued** increase in model performance (loss L).
Such power-law relationship is called a scaling law (read more here[3]).

$$f(x) = ax^{-k} + \epsilon \tag{1}$$

where ϵ denotes a deviation from the power-law function. k is the slope.

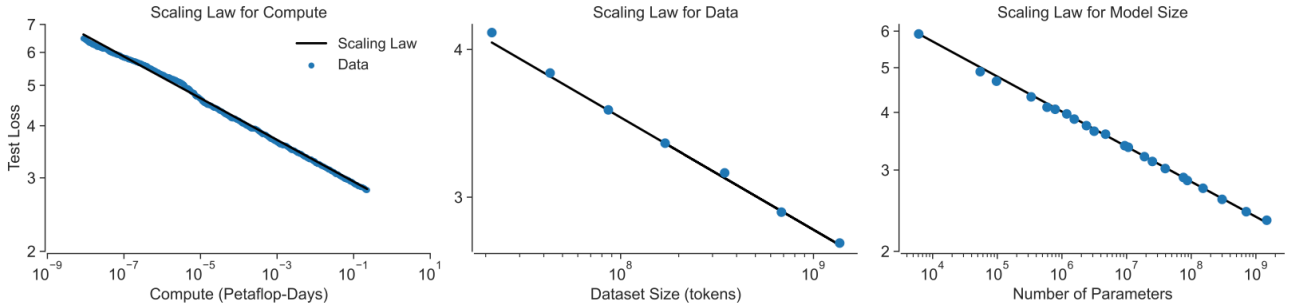


Figure: Scaling laws[1] reliably predict that model performance (y-axes) improves with increasing compute **(Left)**, training data **(Middle)**, and model size **(Right)**. In all cases a power-law (straight line, black) fits the empirically observed data (blue) exceptionally well. Figure adapted from 2. Note that x-axis and y-axis are logarithmic.

---

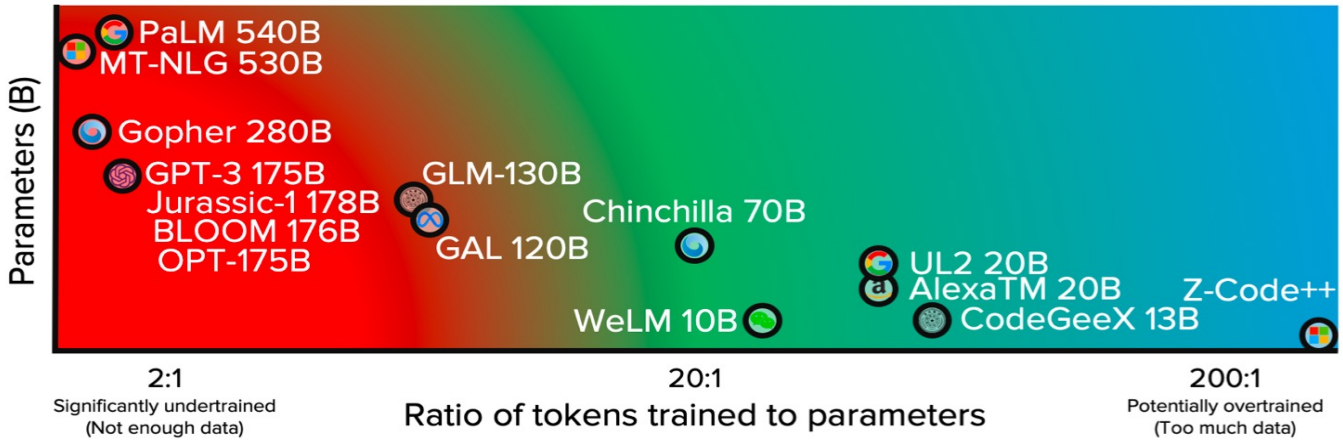
1) Ganguli, Deep, et al. "Predictability and surprise in large generative models." Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency. 2022.

2) Kaplan, Jared, et al. "Scaling laws for neural language models." arXiv preprint arXiv:2001.08361 (2020).

3) `https://en.wikipedia.org/wiki/Neural_scaling_law`

Scaling laws for LLMs

Chinchilla (scaling) law[1,2] suggests we need 20 text tokens per parameter, i.e., 1.4T tokens for training a 70B model.



Is scaling of $D \uparrow$, $C \uparrow$, $N \uparrow$ unlimited, as it's predictable?
By one estimate[2], the world's total stock of usable text data is between 4.6T and 17.2T tokens.

---

1) `https://lifearchitect.ai/chinchilla/`

2) `https://www.lesswrong.com/posts/6Fpvch8RR29qLEWNH/chinchilla-s-wild-implications`

2) Villalobos, Pablo, et al. "Will we run out of data? An analysis of the limits of scaling datasets in Machine Learning." arXiv preprint arXiv:2211.04325 (2022).

Emergent abilities of LLMs

Abrupt jumps in certain known capabilities when increasing model size, same for unknown capabilities which are appeared to be unpredictably **emergent**[1].

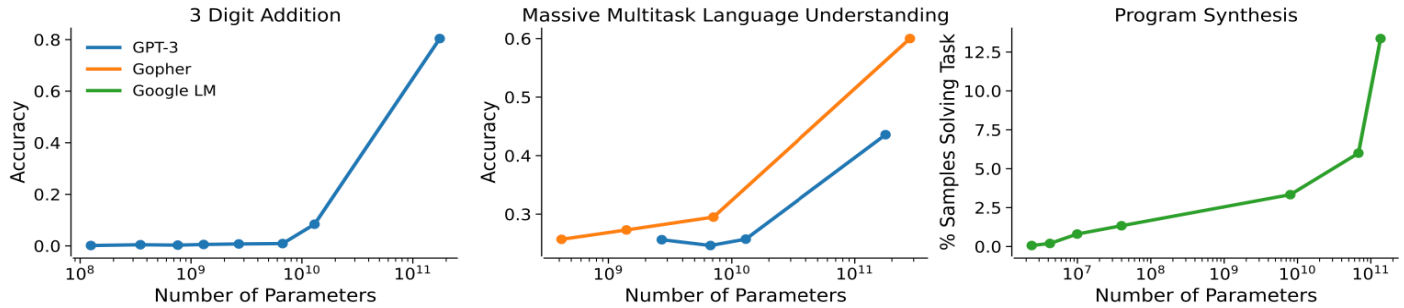


Figure: Three examples of abrupt specific capability scaling, based on three different models: GPT-3 (blue), Gopher (orange), and a Google language model (green). **(Left)** 3-Digit addition with GPT-3. **(Middle)** Language understanding with GPT-3 and Gopher. **(Right)** Program synthesis with Google language models.

Three digit addition is performed accurately less than 1% of the time on any model with less than 6B parameters, but this jumps to 8% accuracy on a 13B parameter model and 80% accuracy on a 175B parameter model.

AI safety: societally impactful potential harms (e.g., toxicity) of the model also scale.

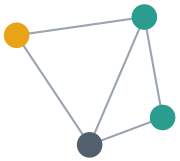
---

1) Wei, Jason, et al. "Emergent abilities of large language models." arXiv preprint arXiv:2206.07682 (2022).

Supervised Fine-Tuning (SFT)

From a text predictor to an assistant that follows instructions

Michalis Vazirgiannis · École Polytechnique / DaSciM



Where SFT sits in the LLM pipeline



1 Pretraining

Objective
next-token prediction on raw text

Scale
trillions of tokens · weeks on thousands of GPUs

What the model gains
knowledge, language, reasoning patterns

2 Supervised fine-tuning

Objective
next-token prediction on (instruction, response) pairs

Scale
10^3–10^5 curated examples · hours to days

What the model gains
format, following instructions, tone

3 Preference optimisation

Objective
RLHF (reward model + PPO) or DPO on ranked answers

Scale
10^4–10^6 comparisons

What the model gains
helpfulness, harmlessness, honesty

Same model, same prompt: “What is the capital of France?”

Base model
“What is the capital of Germany? What is the capital of Italy? …”

After SFT
“The capital of France is Paris.”

The data: conversations, and a loss only on the answer



One training example

```
{"messages": [
  {"role": "system",
   "content": "You are a helpful tutor."},
  {"role": "user",
   "content": "Capital of France?"},
  {"role": "assistant",
   "content": "Paris."} ]}
```

After the chat template: one token sequence

<|system|>	You	are	a	tutor	<|user|>	Capital	of	France	?

<|assistant|>	**Paris**	**.**	**<|end|>**

label = −100: no loss

loss computed here

The objective: the pretraining loss, restricted to the response

$$L(\theta) = -\sum_{(x,y)\in D} \sum_{t=1}^{|y|} \log p_\theta(y_t \mid x, y_{<t})$$

x = prompt (system + user turns), y = assistant response. Nothing new in the architecture or the loss: what changes is the data, and which tokens count.

Where the data come from: quality beats quantity



Human-written

InstructGPT (2022)

≈ 13k demonstrations by trained labellers

high quality, expensive, slow

Synthetic / distilled

Alpaca, self-instruct (2023)

52k instructions generated by a stronger model

cheap and diverse; inherits teacher errors and licences

Small and curated

LIMA (2023)

1,000 carefully selected examples

"knowledge comes from pretraining; SFT teaches format"

What makes a good SFT set

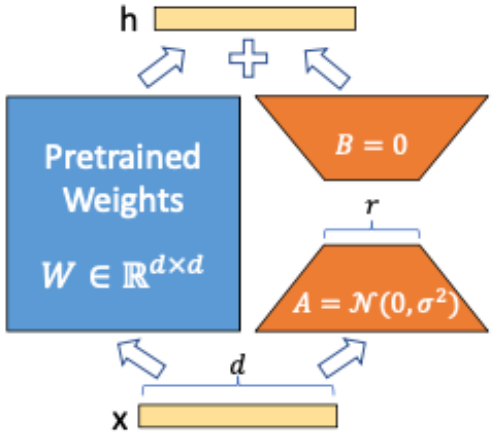
- Correct answers: the model imitates errors as faithfully as good answers
- Diversity of tasks, lengths and difficulty, not 10,000 near-duplicates
- Consistent format and tone: that is what the model will reproduce
- Decontamination: no test-set questions in the training data

Rule of thumb

SFT mostly teaches how to answer, not what is true. If the base model does not know a fact, SFT on that fact teaches it to answer confidently anyway: a known source of hallucination.

LoRA

- ***Low-Rank Adaptation***[1] freezes the pre-trained model weights *W* and injects trainable rank decomposition matrices *A*, *B* into each layer of the Transformer ⇒ *W* + *BA*.
- the learned over-parametrized models in fact reside on a **low intrinsic dimension** [2,3].
- Assume change in weights during model adaptation also has a low "intrinsic rank".

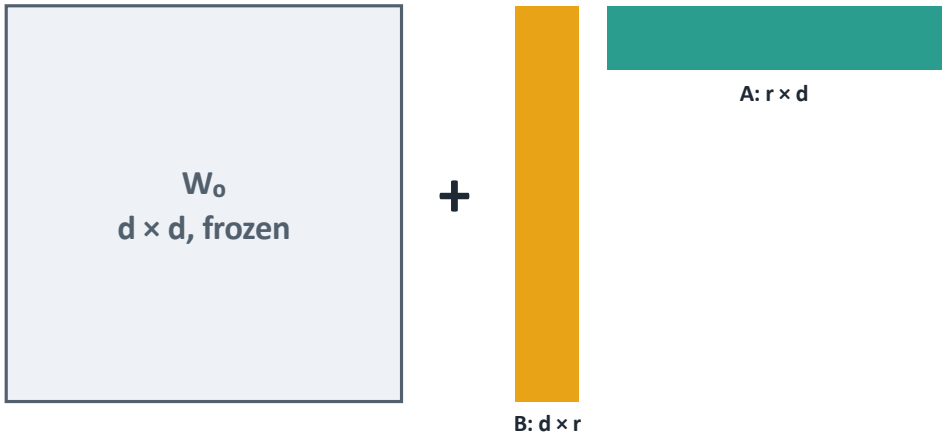


The re-parametrization. We only train *A* and *B*. ***r* can be one or two.**

Weights trained for A and B: $2dr << d^2$ *if* $r << d$

1) Hu, Edward J., et al. "Lora: Low-rank adaptation of large language models." arXiv preprint arXiv:2106.09685 (2021).
2) Li, Chunyuan, et al. "Measuring the intrinsic dimension of objective landscapes." arXiv preprint arXiv:1804.08838 (2018).
3) Aghajanyan, Armen, Luke Zettlemoyer, and Sonal Gupta. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning." arXiv preprint arXiv:2012.13255 (2020).

Fine-tuning cheaply with LoRA



$$h = W_0 x + (\alpha / r) \cdot B A x, \quad r \ll d \ \text{(e.g. } r = 8, d = 4096)$$

- B starts at zero: training starts exactly from the pretrained model
- After training, B A can be merged into W_0: no extra inference cost

	Full fine-tuning	LoRA	QLoRA
Trained parameters	100 %	≈ 0.1–1 %	≈ 0.1–1 %
Base weights stored as	16-bit, updated	16-bit, frozen	4-bit, frozen
GPU memory, 7B model (rough)	> 100 GB	≈ 16–20 GB	≈ 6–10 GB
Output	a new full model	a small adapter (MB)	a small adapter (MB)
Typical learning rate	1e-5 – 2e-5	1e-4 – 2e-4	1e-4 – 2e-4

Memory figures are orders of magnitude; they depend on sequence length, batch size and optimiser.

In practice, and what SFT cannot do



```
from datasets import load_dataset
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer

data = load_dataset("json", data_files="chat.jsonl")

trainer = SFTTrainer(
    model="<base-model-id>",
    train_dataset=data["train"],
    args=SFTConfig(num_train_epochs=2,
                   learning_rate=2e-4,
                   assistant_only_loss=True),
    peft_config=LoraConfig(r=16, lora_alpha=32),
)
trainer.train()
```

Exact option names vary between TRL versions; check the documentation of the installed version.

Limits of SFT

- **Imitation only**
  never sees a bad answer, so cannot learn what to avoid
- **Bounded by the demonstrations**
  rarely better than the people or the model that wrote them
- **Can increase hallucination**
  training on facts the model does not know teaches confident guessing
- **Forgetting**
  narrow SFT can degrade general skills; mix in general data

Next: preference optimisation
RLHF and DPO learn from comparisons of answers: “A is better than B”.

THANK YOU!

http://www.lix.polytechnique.fr/dascim